Ministry of Higher Education and Scientific Research

University of Tunis

Tunis Business School

END-OF-STUDIES INTERNSHIP REPORT

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Science in Business Administration

TITLE

Design and Implementation of an Automated Reporting System for the Huawei Enterprise Business Group (EBG)

AUTHOR

Maysoun Rouached

Academic Supervisor

Dr. Eng. Manel ABDELKADER

Professional Supervisor

Mrs. Khaoula Khemiri

Academic Year: 2025–2026

APPROVAL

APPROVED BY:

ACADEMIC SUPERVISOR

Name Signature Date

PROFESSIONAL SUPERVISOR

Name Signature Date

ACADEMIC EVALUATOR

Name Signature Date

Declaration

I hereby declare that this end-of-studies internship report, entitled *"Design and Implementation of an Automated Reporting System for the Huawei Enterprise Business Group (EBG)"*, is the result of my own work carried out during my internship at Huawei Tunisia, under the supervision of Dr. Eng. Manel Abdelkader and Mrs. Khaoula Khemiri.

I confirm that this report has not been submitted, in whole or in part, for any other academic qualification at this or any other institution, and that all sources of information used in this report have been duly acknowledged and referenced.

Where confidential business data was involved, it has been anonymized or omitted in accordance with Huawei's data confidentiality requirements, without altering the technical substance or validity of the work presented.

Maysoun Rouached

Abstract

This report presents the design and implementation of an automated reporting system developed during an end-of-studies internship within the Channel Team of Huawei Tunisia's Enterprise Business Group (EBG). Prior to this project, the team's recurring reporting activities relied on a fully manual process – extracting, cleaning, consolidating, and formatting data from multiple Excel-based sources – which required approximately one full working day per reporting cycle and was exposed to inconsistency and human error as data volumes grew.

The proposed solution consists of four independent Python-based ETL pipelines, covering Weekly Partners Performance, Channel Revenue, eSales, and Certifications reporting, complemented by a shared graphical user interface, historical snapshot mechanisms for period-over-period comparison, and Excel-based dashboards preserving the reporting format already familiar to the business team. Rather than introducing a centralized data warehouse, the solution was deliberately designed as a lightweight, script-driven ETL architecture built around the team's existing Excel-based workflow, consistent with its current scale, partner base, and weekly reporting cadence.

The automated outputs were validated over approximately one month against previously manually-prepared reports, with discrepancies identified and resolved iteratively in collaboration with the Channel Team. By the end of the internship, the system had fully replaced the manual process across all four reporting domains, reducing the time required to prepare the complete weekly reporting package from approximately one working day to a maximum of approximately 30 minutes, while improving consistency, traceability, and the overall reliability of the resulting reports.

Keywords: Business Intelligence, ETL, Reporting Automation, Python, Excel, Data Quality, Decision Support

Dedication

To my parents, Fatma and Mohamed,

To my mother, Fatma, the safest place I know. For sharing every moment of my life with me, even the smallest, simplest days, my favorite part of any day was always sitting with you and telling you about it. For sacrificing anything for us, and for being the support everyone around you leans on.

To my father, Mohamed, for showing me exactly what love looks like. For sacrificing everything and never once saying no to what I asked, even when you weren't fully convinced. Growing up and away from you was never easy, but your pride in everything I achieve has always made it worth it.

Thank you both for being in my life, I could never dream of a family better than you. May this modest work be the fulfillment of your many wishes and the fruit of your countless sacrifices.

To my brothers,

To Hedi, for always being the first to answer, no matter the time, whenever I was in trouble, without ever thinking of himself first. For believing in me so strongly that if I ever become arrogant, he'll know exactly who's to blame and for always bothering and annoying me.

To Youssef, my little one, for bringing me joy with his endless stories and always wanting to play, and for pushing me to become better, if only so I could finally buy him his simple wish: an iPhone and a PS5.

To my friends,

Who have supported and encouraged me throughout this journey. Especially my roommates, Hadil and Ranim. Thank you for caring about my PFE more than I did myself, and for making this year full of laughter, embarrassing moments, and outings we planned but never actually did.

To Nour Elloumi, thank you for always being there, for the warm welcome and delicious food every time I visit, and for being a truly safe place to just be myself, meeting my constant annoying with nothing but patience.

To all my teachers,

Who contributed to my education throughout my academic path.

To everyone who helped me, directly or indirectly, in accomplishing this work.

Acknowledgment

I would like to express my sincere gratitude to everyone who supported me throughout this internship and the preparation of this report.

I would first like to thank my academic supervisor, Dr. Eng. Manel Abdelkader, for her guidance and feedback throughout this project.

I would also like to thank my professional supervisor at Huawei, Mrs. Khaoula Khemiri, for her support, availability, and valuable feedback throughout my internship, particularly regarding the design and presentation of the reporting dashboards developed during this project. Beyond her technical guidance, she also played an important role in my personal development during this experience, teaching me how to communicate effectively with partners and navigate professional interactions with confidence. Her mentorship made this internship not only a valuable learning experience but also a genuinely enjoyable one, and I am sincerely grateful for the time and patience she invested in me.

My thanks also go to the members of the Channel Team at Huawei Tunisia, for welcoming me into the team and for their collaboration during the validation of the automated reporting system.

Finally, I would like to thank Tunis Business School and its faculty for the academic foundation that made this internship and this project possible.

Table of Contents

APPROVAL **I**

Declaration **II**

Abstract **III**

Dedication **IV**

Acknowledgment **V**

List of Figures **XII**

List of Tables **XIII**

1 Company Presentation and Internship Context **1**

1.1 Introduction 1

1.2 Presentation of Huawei 1

1.3 Huawei's Business Structure 2

1.4 Enterprise Business Group (EBG) 2

1.4.1 EBG Overview 2

1.4.2 EBG Business Activities 3

1.4.3 Channel Team 4

1.5 Huawei Tunisia 5

1.6 My Internship Role 5

1.6.1 Partner Enablement 5

1.6.2 Reporting and Data Analysis 7

1.7 Contribution and Impact 7

1.7.1 Partner Enablement and Event Coordination 8

1.7.2 Reporting and Business Support . . . 8
1.7.3 Automated Reporting System . . . 8
1.7.4 Supporting Excel Tools . . . 9
1.7.5 Overall Impact on the Channel Team . . . 10
1.8 Challenges and Lessons Learned . . . 10
1.9 Project Context and Motivation . . . 11
1.10 Conclusion . . . 11

2 Literature Review and Theoretical Framework 12

2.1 Introduction . . . 12
2.2 Business Intelligence . . . 13
2.2.1 Definition of Business Intelligence . . . 13
2.2.2 Business Intelligence and Decision-Making . . . 13
2.2.3 Business Intelligence Value . . . 14
2.3 Business Reporting Systems . . . 14
2.3.1 Business Reporting . . . 14
2.3.2 Traditional Manual Reporting . . . 15
2.3.3 Limitations of Manual Reporting . . . 15
2.4 Reporting Process Automation . . . 15
2.4.1 Concept of Process Automation . . . 15
2.4.2 Automated Reporting . . . 16
2.4.3 Benefits of Reporting Automation . . . 16
2.5 Data Integration and ETL . . . 17
2.5.1 Data Integration . . . 17
2.5.2 The ETL Process . . . 17
2.5.3 ETL in the Proposed Reporting Context . . . 19
2.6 Data Quality and Validation . . . 19
2.6.1 Importance of Data Quality . . . 19
2.6.2 Data Quality Dimensions . . . 20
2.6.3 Data Cleaning and Validation . . . 20
2.7 Key Performance Indicators and Performance Reporting . . . 21
2.7.1 Key Performance Indicators . . . 21
2.7.2 Performance Monitoring . . . 21

2.8 Data Visualization and Reporting Interfaces . . . 22
2.8.1 Data Visualization . . . 22
2.8.2 Dashboards and Reports . . . 22
2.8.3 Visualization in the Context of the Project . . . 22
2.9 Decision Support Systems . . . 23
2.9.1 Decision Support . . . 23
2.9.2 Descriptive and Diagnostic Analysis . . . 23
2.10 Technologies for Automated Reporting . . . 23
2.10.1 Python for Data Processing . . . 23
2.10.2 Pandas . . . 24
2.10.3 Microsoft Excel . . . 24
2.10.4 Batch Automation . . . 25
2.11 Related Work . . . 25
2.11.1 Overview of Existing Research . . . 25
2.11.2 Comparison of Relevant Approaches . . . 26
2.11.3 Research Gap and Positioning of the Project . . . 27
2.12 Conceptual Framework of the Proposed Solution . . . 28
2.12.1 Data Sources . . . 28
2.12.2 Data Extraction . . . 28
2.12.3 Data Transformation and Validation . . . 28
2.12.4 Data Consolidation . . . 28
2.12.5 Automated Reporting . . . 28
2.12.6 Business Use and Decision Support . . . 28
2.13 On the Role of Data Warehousing in This Project . . . 29
2.14 Conclusion . . . 29

3 Analysis and Design of the Proposed Solution 31

3.1 Introduction . . . 31
3.2 Methodology . . . 32
3.3 Analysis of the Existing Reporting Process . . . 33
3.3.1 Current Reporting Workflow . . . 33
3.3.2 Limitations of the Existing Process . . . 34
3.4 Requirements Analysis . . . 35

3.4.1 Requirement Gathering . . . 35
3.4.2 Functional Requirements . . . 36
3.4.3 Non-Functional Requirements . . . 37
3.5 Proposed Solution . . . 38
3.5.1 Solution Overview . . . 38
3.5.2 Overall System Architecture . . . 39
3.6 Design of the Automated Reporting Pipelines . . . 41
3.6.1 Common Pipeline Architecture . . . 41
3.6.2 Weekly Partners Performance Pipeline . . . 42
3.6.3 Channel Revenue Pipeline . . . 43
3.6.4 eSales Pipeline . . . 43
3.6.5 Certifications Pipeline . . . 43
3.7 Data Transformation and Processing . . . 44
3.7.1 Data Extraction . . . 44
3.7.2 Data Cleaning and Standardization . . . 45
3.7.3 Data Integration . . . 45
3.7.4 KPI and Business Rule Processing . . . 46
3.7.5 Historical Snapshots and Comparisons . . . 46
3.8 Reporting and Dashboard Design . . . 47
3.8.1 Excel Reporting Layer . . . 47
3.8.2 Dashboard Development . . . 48
3.8.3 Preservation and Enhancement of Existing Templates . . . 48
3.9 Automation Architecture . . . 48
3.9.1 Python-Based Automation . . . 48
3.9.2 Graphical User Interface . . . 49
3.9.3 Asynchronous Report Execution . . . 49
3.9.4 Error Handling and Execution Feedback . . . 50
3.9.5 Batch Execution . . . 50
3.10 Project Organization and Maintainability . . . 50
3.10.1 Structured Folder Architecture . . . 50
3.10.2 Separation of Processing and Configuration . . . 51
3.10.3 Reusability . . . 51

3.11 Summary of the Proposed Solution . . . 51
3.12 Conclusion . . . 52

4 Implementation **53**

4.1 Introduction . . . 53
4.2 Project Structure . . . 53
4.3 Common Implementation Conventions . . . 55
4.3.1 Configuration Constants . . . 55
4.3.2 Robust File Reading . . . 55
4.3.3 Manual-Value Preservation . . . 56
4.3.4 Historical Snapshots and Period Comparison . . . 57
4.3.5 Report Styling . . . 57
4.4 Weekly Partners Performance Pipeline . . . 58
4.5 Channel Revenue Pipeline . . . 59
4.6 eSales Pipeline . . . 60
4.7 Certifications Pipeline . . . 60
4.8 Graphical User Interface . . . 61
4.8.1 Design Goal . . . 61
4.8.2 Report Auto-Detection . . . 61
4.8.3 Dynamic Configuration Injection . . . 62
4.8.4 Asynchronous Execution and Logging . . . 63
4.8.5 Batch Launcher . . . 63
4.9 Conclusion . . . 64

5 Results and Evaluation **65**

5.1 Introduction . . . 65
5.2 Evaluation Methodology . . . 65
5.3 Time Efficiency . . . 66
5.4 Sample Outputs . . . 66
5.5 Reliability and Data Quality . . . 68
5.5.1 Duplicate and Data-Quality Handling . . . 68
5.5.2 Review Flagging Instead of Silent Assumptions . . . 68
5.5.3 Flagging Stale Records for Cleanup . . . 69

5.5.4 Manual-Value Preservation 69
5.6 Business Use and Adoption 69
5.7 Limitations 72
5.7.1 Dependence on Source File Structure 72
5.7.2 Fixed Partner and Classification Lists 72
5.7.3 No Automated Test Suite 72
5.8 Future Work 72
5.9 Conclusion 73

General Conclusion **74**

References **75**

List of Figures

1.1 Huawei's business ecosystem and main technology solution domains 2
1.2 Huawei's global enterprise partner ecosystem 3

3.1 Overall architecture of the automated reporting system 40
3.2 Common architecture shared by the four reporting pipelines 41

4.1 Project folder structure – one self-contained subfolder per reporting domain 54
4.2 Reporting Suite GUI – Weekly Partners Performance Report screen 62

5.1 Weekly Partners Performance Report – dashboard sheet 67
5.2 Certifications Report – dashboard sheet 68
5.3 eSales pipeline – Account Manager notification view, flagging overdue and stale opportunities 69
5.4 Channel Revenue Report – dashboard sheet with interactive filter panels 71

List of Tables

2.1 Relevant Literature and Contribution to the Proposed Project 26

3.1 Main components of the proposed reporting solution 51

5.1 Reporting preparation time before and after automation 66

Chapter 1

Company Presentation and Internship Context

1.1 Introduction

This chapter presents the organizational and professional context of the internship carried out at Huawei Tunisia. It first introduces Huawei and its main business areas, before focusing on the Enterprise Business Group (EBG) and the Channel Team within which the internship was conducted.

The chapter then presents my internship role, the main activities and responsibilities assigned to me, and the contributions made to the Channel Team. Finally, it introduces the context that led to the development of the automated reporting system presented in this project.

1.2 Presentation of Huawei

Huawei Technologies Co., Ltd. is a global technology company specializing in information and communication technologies (ICT) and related digital solutions [6]. Founded in 1987, the company has developed a broad portfolio covering telecommunications, enterprise technologies, consumer products, cloud services, and digital power solutions.

Huawei operates internationally and works with customers, business partners, and organizations across different industries [5]. Its activities extend beyond the development of individual technological products and include the design and delivery of integrated solutions adapted to different business and technological requirements. The company's activities are organized around several complementary business areas, allowing Huawei to address the needs of individual consumers, telecommunications operators, and enterprise customers.

1.3 Huawei's Business Structure

Huawei's business activities can be broadly organized around three main areas: Consumers, Carriers, and Enterprises. These areas address different categories of customers and business needs while relying on Huawei's broader technology portfolio and partner ecosystem.

As illustrated in Figure 1.1, Huawei's portfolio covers several technology domains, including devices, intelligent automotive solutions, connectivity, computing, cloud, and digital power. These domains support the different business areas and illustrate the breadth of Huawei's technology ecosystem.

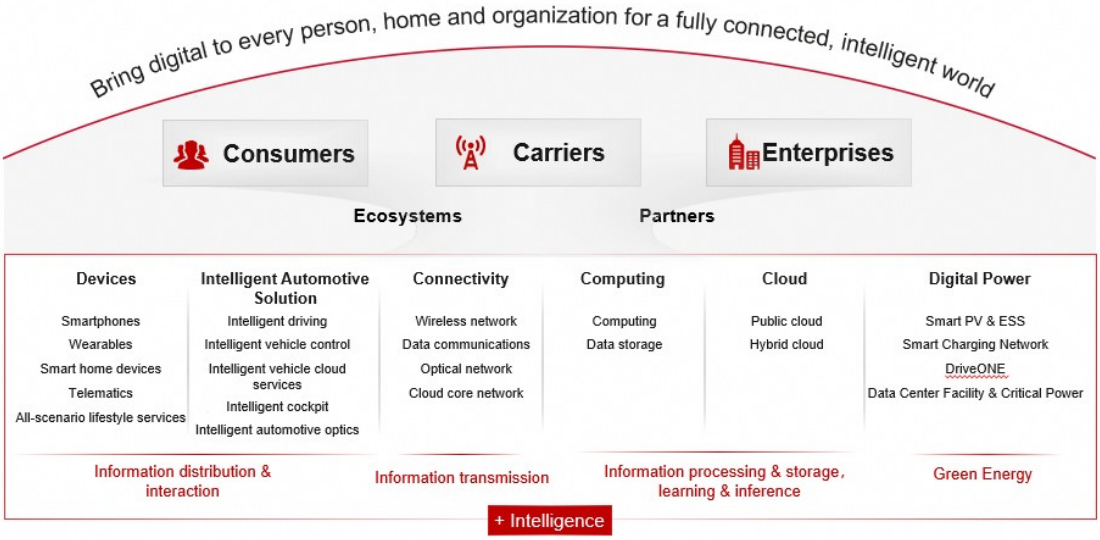


Figure 1.1: Huawei's business ecosystem and main technology solution domains
Source: Huawei partner presentation

The Consumer area focuses on technologies and products intended for individual users, while the Carrier area is associated with telecommunications and connectivity solutions. The Enterprise area addresses the technological requirements of organizations and businesses through a combination of products, solutions, services, and partner collaboration.

Among these three areas, the Enterprise Business Group is particularly relevant to this internship, as the work was carried out within the Enterprise Business Group of Huawei Tunisia.

1.4 Enterprise Business Group (EBG)

1.4.1 EBG Overview

The Enterprise Business Group (EBG) focuses on providing technology products, solutions, and services to enterprise customers across different industries. Its activities combine direct engagement with enterprise customers and collaboration with Huawei's business partner ecosystem.

Within this environment, partners play an important role in promoting Huawei solutions, developing business opportunities, supporting customers, and contributing to the implementation of technology solutions. The EBG therefore relies on continuous coordination between Huawei teams, partners, and customers. As illustrated in Figure 1.2, this partner ecosystem operates at a global scale, spanning tens of thousands of trading partners across industries such as finance, manufacturing, energy, transportation, and public services [7].

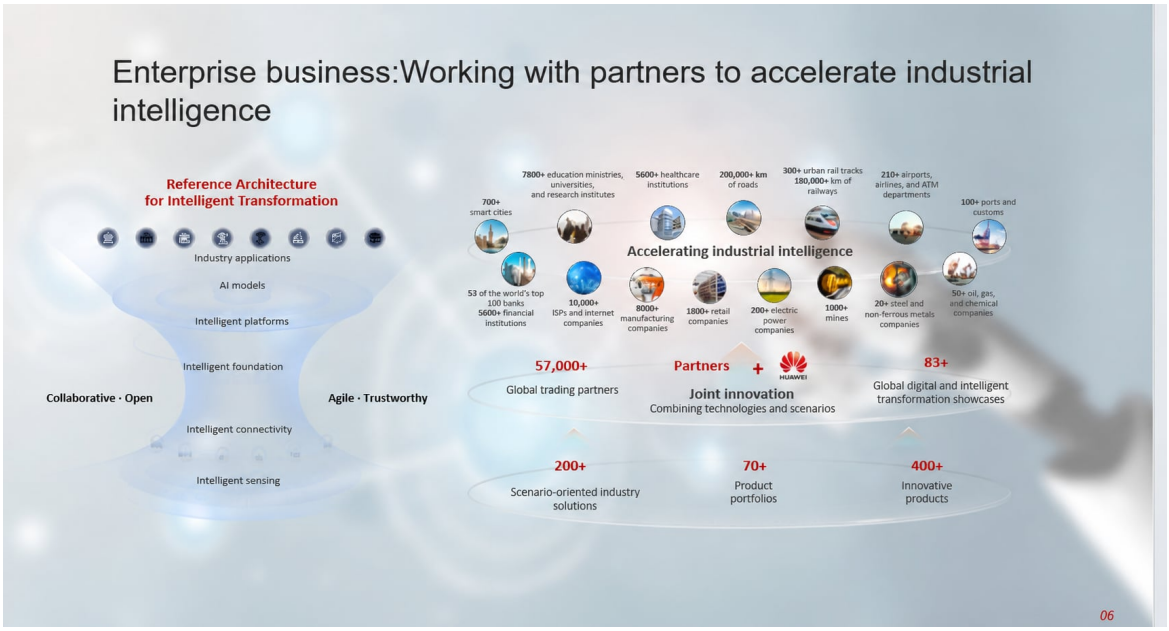


Figure 1.2: Huawei's global enterprise partner ecosystem
Source: Huawei partner presentation

The management of this ecosystem involves several interconnected activities, including sales and order follow-up, partner performance monitoring, incentive management, training and certification, and partner relationship management.

1.4.2 EBG Business Activities

Sales and Order Management

Sales and order management constitute important activities within the EBG. They involve following business opportunities, monitoring partner-related sales activities, and tracking orders throughout the relevant processes.

Accurate and timely information is essential in this context, as sales data must be collected, consolidated, and monitored to provide visibility into business performance.

Partner Performance and Incentives

Partner performance monitoring enables the EBG to evaluate the contribution and development of its partners. Relevant performance indicators can be used to monitor sales results, business growth, and other dimensions of partner activity.

Performance information can also support incentive mechanisms designed to encourage partner engagement and contribute to the achievement of business objectives.

Training and Certification

Training and certification are important components of the EBG's partner ecosystem. They enable partners to develop the technical and commercial capabilities required to promote, sell, and support Huawei solutions.

Monitoring these activities involves following partner training, certification plans, and completed activities, providing visibility into partner enablement and development.

Partner Relationship Management

Partner relationship management involves maintaining communication and coordination with business partners, addressing their needs, and supporting their development within the Huawei ecosystem.

Effective partner relationship management therefore requires continuous coordination and access to reliable information regarding partner activities and performance.

1.4.3 Channel Team

The Channel Team operates within the EBG and focuses on managing and supporting Huawei's channel partners. The team contributes to the development of partner relationships and to the achievement of business objectives through the channel.

Its activities cover several interconnected areas, including sales and performance monitoring, partner enablement, incentive-related activities, training and certification coordination, and partner relationship management.

These activities generate and depend on a significant amount of business information. As a result, reporting and data analysis represent an important part of the team's operational activities.

1.5 Huawei Tunisia

Huawei Tunisia represents the local presence of Huawei within the Tunisian market. The company works with local customers and business partners to support the adoption and deployment of Huawei's technological solutions [4].

Within the Enterprise business, collaboration with partners represents an important part of the local business model. Partners contribute to the promotion, distribution, implementation, and support of Huawei solutions for enterprise customers.

The Channel Team plays an important role in coordinating this partner ecosystem. Its activities require regular communication with partners and continuous monitoring of business, performance, training, and certification-related information.

It is within this organizational context that my internship was conducted.

1.6 My Internship Role

My internship was carried out within the Channel Team of Huawei Tunisia's Enterprise Business Group (EBG), and combined two complementary dimensions: partner enablement and reporting and data analysis. On the operational side, my responsibilities involved certification and training coordination, partner events, and participation in partner meetings. On the analytical side, I worked with the team's existing reporting processes and contributed to their improvement through the development of an automated reporting solution, which is the main technical contribution presented in this project.

Throughout the internship, this combination allowed me to understand how operational activities, partner performance data, and reporting processes are interconnected, and how Business Analytics can contribute to improving decision-making and operational efficiency.

1.6.1 Partner Enablement

Partner enablement was an important part of my responsibilities within the Channel Team. My activities focused on supporting partners in developing their technical capabilities, participating in Huawei programs, and maintaining effective communication between partners and Huawei.

Certification Coordination

One of my main responsibilities was supporting partners in their Huawei certification activities. This involved identifying the certifications required or relevant for each partner according to

the Huawei partner program, including certifications such as HCIA, HCIP, HCIE, and specialist certifications.

For each partner, I contributed to establishing and following a certification plan, coordinating directly with partners to identify their needs, communicate available certification opportunities, and support them in scheduling and completing their examinations. This required regular follow-up to ensure that certification plans progressed according to the partners' needs and the team's objectives. Through this work, the internship contributed to achieving the highest yearly number of partners successfully passing certifications within the scope of my activities.

Training Enablement

I also contributed to partner training activities, coordinating with Huawei technical experts to deliver training sessions to partners in order to strengthen their technical knowledge and product capabilities. I participated in the organization and delivery of one training session alongside a Huawei technical expert, and was involved in monitoring training activities delivered by distributors to their own sub-partners.

These activities allowed me to better understand the importance of continuous technical enablement within the partner ecosystem and the role of training in supporting partners' capabilities and development.

Huawei Partner Skills Contest

One of the most significant partner-related responsibilities entrusted to me was the organization of the local first round of Huawei's annual Partner Skills Contest for the Tunisia–Libya scope.

I was involved in the organization of the activity from the initial preparation phase through to the final ceremony: contacting and inviting participating partners, coordinating with the judges, collecting their requirements regarding evaluation criteria and the type of exercises to be included in the competition, and ensuring the logistical preparation of the contest. I also organized and supported the on-site contest, during which participating partners presented their work and were evaluated by the judging panel, and contributed to the organization of the closing ceremony and the distribution of gifts to the participants and winners.

This activity required extensive coordination across partners, judges, and internal stakeholders, and provided practical experience in managing a multi-stage professional event.

Partner Meetings

I regularly participated in meetings with Huawei partners. My responsibilities included taking notes during meetings, preparing and presenting relevant performance information from the team's reports, and following up on the action items discussed. The reporting information provided during these meetings helped support discussions with partners by presenting their performance and highlighting relevant points that required attention.

Through this activity, I developed a better understanding of partner relationship management and learned how business data can be used to support professional discussions and facilitate decision-making.

1.6.2 Reporting and Data Analysis

Reporting and data analysis represented the second major dimension of my internship. A significant part of my work consisted of understanding the existing manual reporting process and identifying opportunities for improvement and automation.

At the beginning of the project, I focused on understanding how the existing reports were produced, including the different data sources, recurring operations, calculations, and business rules applied by the team. Rather than designing the automated solution independently, I worked closely with the team members responsible for preparing and using the reports, whose practical knowledge of the process was essential for understanding the business requirements and validating the results produced by the automated solution.

The development process was therefore iterative: I first analyzed the existing workflow and identified recurring tasks and pain points, then progressively designed and implemented the automated solution, while continuously comparing its results with the existing reports and incorporating feedback from the team over a validation period of approximately one month. This approach kept the project aligned with actual operational requirements while progressively improving the efficiency and reliability of the reporting process.

1.7 Contribution and Impact

During the internship, I contributed to the Huawei Channel Team through partner enablement activities, certification follow-up, event coordination, recurring reporting, and the development of an automated reporting solution. Beyond supporting the team's daily operations, these contributions helped improve partner engagement, facilitate certification activities, streamline reporting processes, and provide the team with more efficient tools for business monitoring and decision-

making.

1.7.1 Partner Enablement and Event Coordination

I supported the coordination and follow-up of partner training and certification activities by monitoring certification plans, communicating with partners, following up on registration and examination activities, and encouraging partners to complete their certification objectives. This contribution helped strengthen partner engagement with Huawei's certification and enablement programs, and contributed to achieving the highest number of certifications successfully passed by partners during my period of responsibility.

In addition to certification-related activities, I contributed to the organization of the Huawei Partner Skills Contest, including partner communication, coordination with judges, preparation of event materials, logistical organization, and support during the contest day and closing ceremony. This contributed to the smooth execution of the event and strengthened interaction between Huawei and its partner ecosystem.

1.7.2 Reporting and Business Support

I supported the preparation and maintenance of the Channel Team's recurring reports used for partner performance monitoring, business reviews, and internal follow-up. This involved collecting, cleaning, consolidating, and analyzing partner and performance-related information from different sources. The reports provided the team with regularly updated information on partner performance and activities, facilitating the preparation of weekly business reviews and partner meetings.

My contribution also helped identify inconsistencies, missing information, and opportunities for improving the existing reporting workflow. By ensuring that the required information was available in a structured and timely manner, I contributed to reducing the effort required to prepare business reviews and enabled the team to focus more on interpreting results and following up on partner performance rather than on repetitive data preparation tasks.

1.7.3 Automated Reporting System

The main technical contribution of the internship was the design and implementation of an automated reporting system for the Channel Team. The solution automates recurring reporting activities through Python-based reporting pipelines, data processing, KPI calculations, historical comparisons, and Excel report generation. A graphical user interface was also developed to

simplify the execution of the reporting processes and make the solution more accessible to business users.

The implementation significantly improved the efficiency of the reporting workflow. Under the previous manual process, preparing the recurring reports required approximately one working day. With the automated solution, the same reporting activities can be completed in a maximum of approximately 30 minutes, representing a substantial reduction in preparation time. Beyond the time savings, the solution also provides a more standardized reporting process, reduces repetitive manual operations, and limits the risk of calculation and data-processing errors. It therefore provides the Channel Team with a more efficient and reliable approach to recurring reporting and creates additional time for higher-value analysis and decision-making.

The solution was validated over approximately one month following its initial implementation, during which its output was repeatedly compared against the previous manual reports, adjusted, and refined based on feedback from the team, particularly from Mrs. Khaoula Khemiri regarding the design of the dashboard sheets. By the end of the internship, the automated system had fully replaced the previous manual process across all four reporting domains.

1.7.4 Supporting Excel Tools

In addition to the Python-based reporting pipelines, I developed simplified Excel tools based on predefined formulas to facilitate recurring calculations performed by business users, for tasks that did not require full automation.

The **TUNISIA MCR – PRDM Report** addresses the tracking of Account Manager engagement with partners and customers. The underlying data, exported directly from the system, is large in volume, spans Tunisia, Libya, and China, covers a wide range of dates, and contains duplicate entries; it is also split into two distinct datasets, one for PRDM and one for MCR. Previously, producing this report required manually filtering, cleaning, and counting this data for every reporting period. With the Excel tool I built, business users now only need to copy and paste the raw exported data into two designated sheets; the tool then automatically calculates, for each Account Manager, how many times they engaged with a given partner or customer, broken down on a weekly, monthly, quarterly, and yearly basis.

The **Tunisia CM Partner Lead Report** supports the team's partner meetings, during which information such as opportunity amounts and RFP dates is discussed and noted. This information needs to be compared against the corresponding data recorded in the system, in order to verify consistency between what partners report and what is officially tracked. This comparison was previously done manually; the Excel tool I built now only requires the relevant

data to be copied into the sheet, after which the comparison is produced automatically.

Beyond these two reports, these lightweight tools preserved the Excel-based working environment already used by the team, reduced repetitive manual calculations, and allowed users to obtain results more quickly without having to rebuild formulas or repeat manual filtering and comparison work for every reporting period.

1.7.5 Overall Impact on the Channel Team

Taken together, these contributions generated value along several complementary dimensions. Partner enablement and event coordination strengthened engagement with Huawei's training and certification programs; the automated reporting system reduced recurring preparation time from approximately one working day to a maximum of 30 minutes while improving consistency and reliability; and the supporting Excel tools extended these benefits to tasks that did not warrant full automation. Because the Python pipelines and Excel tools were designed to be reusable rather than addressing a one-time need, this impact is not limited to the internship period: the Channel Team retains tools that can be reused and adapted as its reporting requirements evolve.

1.8 Challenges and Lessons Learned

The internship involved several challenges

One of the main challenges was working with reporting processes that involved multiple data sources, recurring calculations, and business-specific requirements. Understanding these processes required careful analysis and regular validation with the team.

Another challenge was ensuring that the automated results remained consistent with the reports previously produced manually. This required repeated testing, comparison, and adjustment during the development of the solution.

The internship also required balancing technical development with operational responsibilities, particularly during periods involving partner activities, meetings, training, and event organization.

These challenges strengthened my ability to analyze business requirements, work with real-world data, communicate with stakeholders, and translate operational needs into practical analytical and technical solutions.

1.9 Project Context and Motivation

The reporting activities performed within the Channel Team revealed the importance of timely and reliable access to partner and business performance information. Before the implementation of the proposed solution, the preparation of recurring reports relied heavily on manual operations, including data extraction, cleaning, filtering, calculations, consolidation, and report preparation.

The large volume of data made this process particularly time-consuming. Although the same types of source files were used regularly, the manual nature of the process required significant effort and limited the practical ability to process and present all available information.

The reporting process could require approximately one full working day to prepare the recurring reports. This created an opportunity to improve the process through automation while preserving the business requirements and reporting formats used by the team.

The project presented in this report was therefore initiated to address this operational need. The objective was to design and implement an automated reporting system capable of processing recurring data, applying the required business rules and calculations, generating reports and dashboards, and facilitating access to performance information.

The resulting project is entitled:

Design and Implementation of an Automated Reporting System for the Huawei Enterprise Business Group (EBG)

1.10 Conclusion

This chapter presented the organizational context of the internship, from Huawei's global business structure to the Enterprise Business Group and the Channel Team within Huawei Tunisia. It also described the responsibilities undertaken during the internship, covering both partner enablement and reporting and data analysis activities.

The internship combined operational involvement with a technical project focused on improving the Channel Team's reporting process. The observation of the existing manual workflow and its limitations led to the identification of an opportunity for automation.

The next chapter reviews the theoretical concepts and existing research related to Business Intelligence, reporting automation, data integration, data quality, performance monitoring, dashboards, and decision support, providing the theoretical foundation for the proposed solution.

Chapter 2

Literature Review and Theoretical Framework

2.1 Introduction

The increasing availability of organizational data has transformed the way companies monitor their activities and support managerial decision-making. However, the availability of data alone does not guarantee that organizations can effectively use it. Data must first be collected, integrated, cleaned, transformed, analyzed, and presented in a form that is relevant to business users.

Business Intelligence (BI) and Business Analytics provide a framework for transforming organizational data into meaningful information that can support operational and managerial decisions. At the same time, the automation of data preparation and reporting processes can reduce repetitive manual activities, improve data consistency, and facilitate timely access to business information.

This chapter presents the theoretical and technical foundations underlying the development of the automated reporting system proposed in this project. It focuses on Business Intelligence, business reporting, reporting process automation, data integration and ETL, data quality, Key Performance Indicators (KPIs), data visualization, decision support, and the technologies used for data processing and automation.

The objective is not to reproduce the complete theoretical landscape of Business Intelligence, but rather to review the concepts that are directly relevant to the problem addressed by this project. The chapter concludes with a review of related work and the positioning of the proposed solution within the existing literature.

2.2 Business Intelligence

2.2.1 Definition of Business Intelligence

Business Intelligence (BI) is an important area within information systems that focuses on transforming organizational data into information that can support business users and decision makers. Negash defines Business Intelligence systems as systems that combine data gathering, data storage, knowledge management, and analytical tools to provide information to planners and decision makers. The objective is to improve the timeliness and quality of the information available for decision-making processes [10].

Similarly, Chen, Chiang, and Storey describe Business Intelligence and Analytics (BI&A) as a major research and practice area concerned with addressing data-related problems in contemporary organizations. Their framework highlights the evolution of BI&A and its applications across different types of data and analytical environments [2].

Business Intelligence can therefore be understood as a combination of processes, technologies, and analytical practices that transform raw organizational data into meaningful information. The general process begins with the collection of data from one or more sources. The data is subsequently integrated and transformed before being analyzed and presented through reports, dashboards, or other analytical outputs. The final objective is to provide information that can support monitoring, analysis, and decision-making.

2.2.2 Business Intelligence and Decision-Making

The value of Business Intelligence is closely related to its ability to improve the information available to decision makers. Organizations often possess large amounts of operational data, but decision makers need structured and relevant information rather than raw data.

BI systems can therefore serve as an intermediary between operational data and managerial decision-making. Instead of requiring users to manually collect information from several sources, BI processes can consolidate information and provide indicators that facilitate the interpretation of organizational performance.

Recent literature also emphasizes the relationship between Business Intelligence, analytics, and decision support. BI and Analytics systems can be viewed as information systems that transform data into deeper insights that can contribute to decision-making processes [13].

For an organization such as Huawei's Channel Team, this principle is particularly relevant because operational information can originate from several reporting sources and must be consolidated before it can be used to monitor partner performance and prepare business meetings.

2.2.3 Business Intelligence Value

The implementation of a BI system does not automatically guarantee organizational value. Research on BI systems has highlighted the importance of how BI technologies are implemented, adopted, used, and integrated into organizational processes.

A systematic review by Trieu identifies BI system adoption, utilization, and success as important areas of research and highlights the organizational and technological challenges associated with obtaining value from BI systems [14]. This indicates that the usefulness of a BI solution depends not only on the technology itself but also on its alignment with organizational needs and user requirements.

Consequently, a Business Intelligence solution should be designed around actual business processes and the information requirements of its users. This principle is particularly relevant to the present project, where the automated reporting system was developed around an existing reporting process and validated through interaction with the team members who use the reports.

2.3 Business Reporting Systems

2.3.1 Business Reporting

Business reporting refers to the process of collecting and presenting organizational information in a structured format for monitoring, analysis, and communication purposes.

Reports can contain numerical indicators, tables, comparisons, trends, and other forms of information that allow users to evaluate organizational activities. Reporting is therefore an important link between operational data and managerial action.

A reporting process generally involves several stages:

1. Collection of data from relevant sources;
2. Preparation and cleaning of the data;
3. Transformation and consolidation of information;
4. Calculation of relevant indicators;
5. Generation of reports;
6. Interpretation and use of the resulting information.

The quality and efficiency of a reporting process therefore depend not only on the final report but also on the activities performed before its generation.

2.3.2 Traditional Manual Reporting

Despite the development of Business Intelligence platforms and automated data-processing technologies, spreadsheet-based reporting remains widely used in organizations.

Manual reporting processes frequently involve extracting information from different sources, opening several files, copying or transferring information, applying formulas, consolidating datasets, checking results, and finally producing a report.

Although this approach can be appropriate for small or occasional reporting requirements, repetitive reporting activities may become inefficient when performed on a weekly or daily basis.

2.3.3 Limitations of Manual Reporting

Manual reporting can create several operational challenges.

First, repetitive data manipulation requires considerable employee time. Activities such as copying information, consolidating files, and recalculating indicators may have to be repeated every reporting period.

Second, manual processes may increase the risk of human error [11]. Errors can occur during data entry, copying, formula modification, filtering, or consolidation.

Third, maintaining consistency can become difficult when the same reporting process is performed repeatedly or by different users.

Finally, manual reporting can limit the time available for analysis. When employees spend a significant proportion of their time preparing information, less time remains available for interpreting the results and supporting business decisions.

These limitations provide a strong motivation for automating repetitive reporting activities.

2.4 Reporting Process Automation

2.4.1 Concept of Process Automation

Process automation refers to the use of technology to execute repetitive and structured activities with limited manual intervention.

In a business context, automation can be applied to activities such as data collection, data processing, calculations, file generation, notifications, and reporting.

The objective is not necessarily to eliminate human involvement completely. Instead, automation can be used to transfer repetitive and rule-based activities from manual execution to a computer-based process, allowing employees to focus on activities that require interpretation,

judgment, communication, and decision-making.

2.4.2 Automated Reporting

Automated reporting applies process automation principles to the generation and preparation of business reports.

An automated reporting process can include:

- Automatic extraction of data from predefined sources;
- Data cleaning and standardization;
- Data transformation and consolidation;
- Automated calculations;
- Data validation;
- Generation of standardized reports;
- Updating reports when new source data becomes available.

The main advantage of such an approach is the reduction of repetitive manual operations. This is consistent with real-world evidence from organizations automating recurring financial and administrative processes, such as the robotic process automation case documented by Asatiani and Penttinen [1]. The automation of reporting processes can also contribute to standardization. Once the transformation rules and calculations are implemented in a controlled process, the same logic can be applied consistently to successive reporting periods.

2.4.3 Benefits of Reporting Automation

The main benefits relevant to organizational reporting include:

- **Time efficiency:** reducing the amount of time required to perform repetitive reporting operations;
- **Consistency:** applying the same processing rules to successive datasets;
- **Reduced human error:** limiting repetitive manual manipulation;
- **Scalability:** allowing the same process to handle recurring datasets without repeating every operation manually;

- **Reproducibility:** producing reports using a defined and repeatable workflow;
- **Improved availability of information:** making processed information available more efficiently to business users.

However, automation does not remove the need for human validation. Automated processes remain dependent on the quality and structure of their input data and on the correctness of the implemented business rules.

Therefore, automation should be combined with data validation and business-user verification.

2.5 Data Integration and ETL

2.5.1 Data Integration

Data integration is the process of combining information originating from different sources into a coherent dataset or information environment.

Organizations frequently store operational information in different systems, files, databases, or applications. These sources may use different structures, naming conventions, formats, and levels of detail.

Consequently, integration requires more than simply bringing files together. The data must be understood, standardized, and transformed so that information from different sources can be combined correctly.

In the context of Business Intelligence, data integration represents an important step between operational data sources and analytical outputs.

2.5.2 The ETL Process

Extract, Transform, and Load (ETL) is a commonly used approach for processing data originating from different sources.

Vassiliadis, Simitsis, and Skiadopoulos studied the logical modeling of ETL processes and described ETL environments as involving activities related to extracting data from sources, cleansing and transforming it, and loading the resulting information into a target environment [15]. This academic treatment is complemented by more practice-oriented guidance on ETL implementation, such as Kimball and Caserta's toolkit for extracting, cleaning, conforming, and delivering data [8].

The three stages can be described as follows.

Extract

The extraction stage consists of obtaining data from one or more source systems.

Sources may include:

- Databases;
- Enterprise information systems;
- APIs;
- CSV files;
- Excel workbooks;
- Other structured or semi-structured sources.

In the context of this project, the source information is exported from Huawei's internal system through different tabs or reporting views. These exports are provided as Excel files and constitute the input datasets of the reporting process.

Transform

The transformation stage consists of preparing the extracted information for further analysis and reporting.

Transformation activities may include:

- Data cleaning;
- Standardization of formats;
- Handling missing or inconsistent values;
- Filtering;
- Joining datasets;
- Aggregation;
- Calculation of indicators;
- Comparison with historical information;
- Validation of business rules.

This stage is particularly important because raw source data is not necessarily organized in the form required by business users.

Load

The loading stage consists of transferring the transformed information into the destination environment where it will be used.

The destination does not necessarily have to be a traditional enterprise data warehouse. Depending on the requirements of the application, transformed information can be loaded into files, databases, analytical platforms, or reporting environments.

In the present project, the processed information is ultimately used to generate comprehensive Excel-based reports. Therefore, the reporting workbook represents the main output environment of the implemented pipeline.

2.5.3 ETL in the Proposed Reporting Context

The reporting process developed in this project can be interpreted as a lightweight ETL-based pipeline. The extraction stage begins with the export of several Excel datasets from Huawei's internal reporting environment. The transformation stage is performed programmatically and includes cleaning, preparation, consolidation, calculations, comparisons, and validation. The resulting information is then used to generate the comprehensive weekly reports.

This architecture demonstrates that ETL concepts can be applied without necessarily implementing a centralized data warehouse. The appropriate architecture depends on the organization's requirements, data volume, infrastructure, and intended use of the processed information.

2.6 Data Quality and Validation

2.6.1 Importance of Data Quality

Data quality is a fundamental requirement for reliable reporting and Business Intelligence.

A reporting system may correctly execute every transformation rule while still producing misleading results if the input data is incomplete, inconsistent, outdated, or incorrectly represented.

Wang and Strong argue that data quality should not be reduced to accuracy alone. Their framework identifies several dimensions of data quality and emphasizes that data should be considered in relation to the needs of its consumers and the context in which it is used [16].

This perspective is particularly relevant to business reporting because information may be technically accurate but still unsuitable for decision-making if it is incomplete, inconsistent, poorly represented, or unavailable when needed.

2.6.2 Data Quality Dimensions

Relevant data quality dimensions include:

- **Accuracy:** whether the data correctly represents the intended value;
- **Completeness:** whether required information is present;
- **Consistency:** whether information follows coherent values and structures across datasets;
- **Timeliness:** whether information is available at the required time;
- **Validity:** whether values conform to the expected format or business rules;
- **Uniqueness:** whether duplicate records are appropriately controlled.

These dimensions are important when integrating multiple reporting files because differences between source datasets can affect the reliability of the final report.

2.6.3 Data Cleaning and Validation

Data cleaning consists of identifying and addressing problems within datasets before they are used for analysis.

Typical operations include:

- Removing unnecessary records;
- Standardizing names and formats;
- Handling missing values;
- Detecting duplicates;
- Converting data types;
- Checking numerical values;
- Verifying relationships between datasets.

Validation then ensures that the transformed information follows the expected business logic.

For an automated reporting system, validation is particularly important because automation can reproduce an incorrect rule consistently. Therefore, automated outputs should be compared against expected results and validated with business users.

2.7 Key Performance Indicators and Performance Reporting

2.7.1 Key Performance Indicators

Key Performance Indicators (KPIs) are measurable indicators used to monitor organizational performance in relation to defined objectives.

KPIs allow large amounts of operational information to be summarized into indicators that are easier to interpret and monitor.

A useful KPI should be relevant to the business objective it represents and should provide information that can support monitoring or action.

In a partner-management environment, examples of performance indicators may include revenue, target achievement, revenue evolution, partner activity, certification progress, or other measures defined by the organization.

The selection of KPIs should therefore be based on the actual information requirements of the business users rather than on the availability of data alone.

2.7.2 Performance Monitoring

Performance reporting involves comparing current information with targets, previous periods, or other relevant reference points.

For example, a reporting system can allow users to identify:

- Whether a partner is meeting a target;
- Whether performance has increased or decreased;
- Which partners require attention;
- How current performance compares with historical performance.

Such comparisons transform isolated values into contextual information that is more useful for business analysis.

This principle is directly relevant to the reporting process addressed in this project, where different datasets are consolidated to provide a comprehensive view of partner and channel performance.

2.8 Data Visualization and Reporting Interfaces

2.8.1 Data Visualization

Data visualization consists of representing information graphically or visually in order to facilitate interpretation.

When datasets become large or contain multiple dimensions, visual representations can help users identify patterns, comparisons, trends, and anomalies more rapidly than raw tables alone.

The effectiveness of visualization depends on selecting an appropriate representation for the information being communicated.

2.8.2 Dashboards and Reports

Dashboards are designed to provide users with a concise view of important information, often with the objective of enabling rapid monitoring.

Few emphasizes that effective dashboards should communicate relevant information clearly and rapidly and that visual design should support the user's ability to interpret information efficiently [3].

Reports and dashboards serve related but distinct purposes. A report may provide detailed structured information, while a dashboard generally emphasizes high-level monitoring and rapid interpretation.

The choice between a detailed report, a dashboard, or a combination of both should therefore depend on the user's information requirements.

2.8.3 Visualization in the Context of the Project

The objective of the reporting system developed in this project is primarily to consolidate and automate recurring business information. Consequently, the presentation layer focuses on structured and comprehensive reporting rather than requiring a completely independent visualization platform.

The theoretical principles of visualization remain relevant because the final report must present information in a clear and usable manner. Performance indicators, comparisons, trends, and other relevant information should be organized so that users can identify the information required for operational follow-up.

2.9 Decision Support Systems

2.9.1 Decision Support

Decision Support Systems (DSS) are information systems designed to support decision-making by integrating data, analytical models, and domain knowledge.

The relationship between Business Intelligence (BI), Analytics, and Decision Support Systems has been widely examined in information systems research. Phillips-Wren and colleagues [13] contend that BI and Analytics constitute a field closely aligned with decision support, sharing the common objective of converting growing volumes of data into deeper, actionable insights.

This transformation can be summarized as:

Raw Data → Information → Analysis → Insight → Decision Support

2.9.2 Descriptive and Diagnostic Analysis

The reporting system developed in this project is primarily associated with descriptive and diagnostic analytics.

Descriptive analytics focuses on understanding what has happened based on historical or current data.

For example, a report may indicate the revenue generated by a partner during a given period.

Diagnostic analytics goes further by examining differences, comparisons, and potential explanations for observed results.

For example, comparing a partner's current revenue with its previous performance can highlight an increase or decrease that requires further attention.

Predictive and prescriptive analytics are not considered central components of the current project because the primary objective is the automation and consolidation of existing reporting activities rather than forecasting or automated recommendation generation.

2.10 Technologies for Automated Reporting

2.10.1 Python for Data Processing

Python is widely used for data manipulation and analysis because of its extensive ecosystem of libraries and its suitability for automating data-processing tasks.

McKinney's work on Python for Data Analysis presents Python and the pandas ecosystem as tools for manipulating, cleaning, transforming, combining, and analyzing structured datasets [9].

In the context of automated reporting, Python can be used to automate repetitive data-processing activities that would otherwise require manual manipulation.

The role of Python in the proposed system is therefore primarily associated with:

- Reading input files;
- Processing datasets;
- Cleaning and transforming information;
- Combining datasets;
- Performing calculations;
- Comparing information across reporting periods;
- Preparing outputs for reporting.

2.10.2 Pandas

The pandas library provides data structures and operations designed for practical work with structured datasets. McKinney introduced pandas as a library intended to facilitate working with datasets and provide building blocks for data analysis and statistical computing [9].

Its functionality is particularly relevant to the present project because the reporting workflow requires operations such as loading, cleaning, merging, grouping, aggregating, and transforming tabular data.

2.10.3 Microsoft Excel

Excel is an important component of the reporting environment addressed by this project.

Spreadsheet applications are commonly used by organizations for data manipulation, calculations, reporting, and presentation. In the present context, Excel plays two roles:

1. It represents the format in which data is exported from the internal reporting environment;
2. It represents the principal format in which the resulting comprehensive reports are delivered.

Therefore, the objective of the proposed automation is not to eliminate Excel from the reporting process, but to reduce the manual operations required to prepare Excel-based reports.

2.10.4 Batch Automation

The execution of a reporting process can also be automated at the operating-system level.

A batch execution mechanism can allow several processing operations or Python scripts to be launched through a predefined command rather than executed individually.

In the context of the project, this approach contributes to making the reporting workflow more repeatable and easier to execute.

2.11 Related Work

2.11.1 Overview of Existing Research

The literature surrounding Business Intelligence has developed considerably and covers topics including BI architectures, analytical systems, decision support, data integration, visualization, adoption, and organizational value. Read together, these strands form a layered picture: foundational definitions of what BI systems are and do [10, 2]; process-level work on how data actually moves through such a system, from source to insight [15, 16]; and a more recent, adoption-focused strand asking why BI implementations succeed or fail in practice regardless of their technical soundness [14, 13]. Visualization research [3] sits somewhat apart from this progression, addressing the final, user-facing layer common to all of the above.

Negash's definition of BI as the combination of data gathering, storage, knowledge management, and analytical tools [10] and Chen, Chiang, and Storey's broader BI&A framework [2] agree on the same basic premise – that BI's value lies in transforming raw data into decision-relevant information – but differ in scope: Negash's definition is system-centric, describing what a BI system contains, while Chen et al. treat BI&A as a research field defined by the problems it addresses (large, heterogeneous data; the need for actionable insight) rather than by a fixed architecture. This project's own scope sits closer to Negash's system-level framing, since it concerns one specific reporting system rather than a research agenda.

Where these definitional works describe *what* a BI system should do, Vassiliadis, Simitsis, and Skiadopoulos's ETL modeling work [15] and Wang and Strong's data quality framework [16] describe *how* it does it reliably. The two are complementary rather than overlapping: ETL modeling formalizes the movement of data from source to target, while the data quality framework insists that movement alone is not sufficient – a technically correct ETL pipeline can still produce reports that are inaccurate, inconsistent, or untimely if data quality is not addressed as a separate concern. Both perspectives were directly relevant to this project, since the pipeline's data-cleaning and validation steps (Chapter 3) exist precisely to address the kind of quality issues Wang and

Strong describe, using the extract-transform-load structure Vassiliadis et al. formalize.

A third strand, represented by Trieu's review of BI adoption, utilization, and success [14] and Phillips-Wren, Daly, and Burstein's work connecting BI, Analytics, and Decision Support Systems [13], shifts the question from technical correctness to organizational fit. Trieu's central finding – that BI value depends as much on adoption and alignment with user needs as on the technology itself – is the reason this project was developed iteratively with the Channel Team rather than designed in isolation and delivered as a finished system (Chapter 1). Phillips-Wren et al.'s framing of BI and Analytics as inherently connected to decision support reinforces the same point from a different angle: a reporting system's purpose is not the report itself, but the decisions it enables, which is why usability and preservation of the team's existing Excel-based workflow (Chapter 3) were treated as design requirements rather than implementation details.

Finally, Few's work on dashboard design [3] addresses the layer where all of the above becomes visible to the end user: even a well-modeled, high-quality, well-adopted BI process can fail at the point of communication if its output is not organized for rapid interpretation. This is the direct justification for the in-place visual cues (conditional formatting, color-coded deltas) described in the design of this project's reporting layer (Chapter 3, Section 2.7.2), rather than leaving the reader to compare raw numbers unaided.

2.11.2 Comparison of Relevant Approaches

The following table summarizes the main concepts from the literature that are relevant to the proposed project.

Table 2.1: Relevant Literature and Contribution to the Proposed Project

Reference	**Main Focus**	**Main Contribution**	**Relevance to the Project**
Negash (2004)	Business Intelligence	Defines BI as the combination of data gathering, storage, knowledge management, and analytical tools to support decision makers.	Provides the BI foundation for the reporting system.
Chen et al. (2012)	Business Intelligence and Analytics	Provides a framework for BI&A research and its evolution, applications, and challenges.	Positions the project within the broader BI&A field.

Reference	Main Focus	Main Contribution	Relevance to the Project
Vassiliadis et al. (2002)	ETL processes	Examines the modeling of extraction, transformation, and loading processes.	Provides the theoretical basis for the project's data-processing pipeline.
Wang and Strong (1996)	Data Quality	Shows that data quality includes multiple dimensions beyond accuracy.	Supports the need for validation and consistency checks.
Trieu (2017)	BI System Value	Reviews research on BI adoption, utilization, success, and organizational value.	Highlights the importance of aligning the solution with business users and processes.
Few (2013)	Dashboard Design	Provides principles for clear and effective presentation of business information.	Supports the design and presentation of reporting outputs.

Source: Author

2.11.3 Research Gap and Positioning of the Project

The reviewed literature demonstrates that Business Intelligence research frequently addresses broad BI architectures, analytical capabilities, decision support, data integration, organizational adoption, and visualization.

However, the needs of an operational reporting environment are not necessarily identical to those of a large-scale enterprise BI architecture. In particular, organizations may already possess operational systems and established reporting formats while still relying on repetitive manual activities to extract, consolidate, and prepare information.

This project addresses this practical perspective by focusing on the automation of an existing reporting workflow rather than replacing the organization's underlying systems.

The proposed solution is therefore positioned as a lightweight and business-oriented reporting automation approach. It uses existing Excel exports as source data, applies programmatic data processing and transformation, consolidates information from multiple sources, and generates comprehensive reporting outputs.

The project does not attempt to implement a full enterprise data warehouse or replace Huawei's internal systems. Instead, it focuses on improving the efficiency, consistency, and repeatability of an existing operational reporting process.

2.12 Conceptual Framework of the Proposed Solution

Based on the concepts reviewed above, the proposed solution can be represented through the following conceptual framework, consisting of six main stages.

2.12.1 Data Sources

The process begins with information exported from Huawei's internal reporting environment. Different tabs or reporting views provide different Excel datasets.

2.12.2 Data Extraction

The exported Excel files are collected and made available to the automated processing pipeline. These files constitute the raw inputs of the reporting process.

2.12.3 Data Transformation and Validation

The raw datasets are processed programmatically. The transformation stage includes cleaning, formatting, consolidation, calculations, comparisons, and validation according to the reporting requirements.

2.12.4 Data Consolidation

The processed information is combined into a coherent structure that provides a comprehensive view of the information required by the Channel Team.

2.12.5 Automated Reporting

The consolidated information is used to generate the required weekly reports. Automation ensures that recurring processing operations follow a defined and repeatable workflow.

2.12.6 Business Use and Decision Support

The resulting reports are used by business users for performance monitoring, partner meetings, business reviews, and operational follow-up.

The framework therefore connects the technical dimension of the project with its business objective:

Data Integration → Automation → Reporting → Performance Monitoring → Decision Support

2.13 On the Role of Data Warehousing in This Project

Because data warehousing is closely associated with BI and ETL in the literature, it is worth explicitly addressing why this project does not implement one. Chaudhuri and Dayal (1997) characterize data warehousing and OLAP as decision-support technologies designed for environments where large volumes of historical, multi-source data must be queried flexibly by many analysts, often along multiple dimensions (time, geography, product). These requirements justify the engineering cost of a dimensional data model, ETL jobs feeding a persistent store, and a query layer on top of it.

The Channel Team's reporting process does not currently exhibit these characteristics: the partner base is small and known in advance, the reporting cadence is weekly rather than continuous, the primary consumer is a single team producing a single comprehensive report rather than many analysts issuing ad-hoc queries, and the final output, an Excel workbook, is itself the interface the business already trusts and knows how to use. Introducing a relational data warehouse, a star schema, and a separate BI front end onto this workflow would add architectural complexity without a corresponding business requirement, and would be difficult to justify. The more defensible and academically accurate framing of this project is therefore an Excel-based, script-driven ETL pipeline for automated business reporting, rather than a data-warehouse implementation.

2.14 Conclusion

This chapter established the theoretical and technical foundations of the proposed automated reporting system.

The literature review demonstrated that Business Intelligence aims to transform organizational data into useful information for business users and decision makers. Reporting systems provide a structured mechanism for communicating this information, while automation can reduce repetitive manual operations and improve the consistency and repeatability of reporting processes.

The review of ETL concepts provided a framework for understanding the data-processing pipeline implemented in this project. The extraction of data from multiple Excel exports, followed by cleaning, transformation, consolidation, validation, and report generation, represents a practical application of ETL principles.

Data quality was also identified as an essential consideration because the reliability of reporting outputs depends on the quality and consistency of the underlying data. Finally, the discussion of KPIs, visualization, and decision support demonstrated how processed information can be

transformed into useful business information for performance monitoring and operational decision-making.

Based on these theoretical foundations, the project is positioned as an automated, ETL-based reporting solution designed around an existing operational process. Rather than introducing unnecessary technological complexity, the proposed approach focuses on addressing the actual reporting requirements of the Channel Team through data integration, process automation, validation, and structured reporting.

The following chapter therefore moves from the theoretical foundations to the analysis of the existing reporting process, its limitations, user requirements, and the specifications that guided the design of the proposed solution.

Chapter 3

Analysis and Design of the Proposed Solution

3.1 Introduction

The previous chapter established the theoretical foundations of Business Intelligence, reporting automation, data integration, ETL processes, data quality, Key Performance Indicators (KPIs), and dashboard design. These concepts provide the theoretical basis for the solution developed during this project.

Within Huawei's Enterprise Business Group (EBG), reporting activities rely on data originating from different internal sources and exported in different formats. Before the implementation of the proposed solution, the preparation of recurring reports involved several manual operations, including data extraction, cleaning, filtering, calculations, consolidation, and report updating. Although the process was based on recurring and relatively structured tasks, its manual nature resulted in significant preparation time and limited the ability to efficiently process large volumes of data.

The objective of this project was therefore to design and implement an automated reporting system capable of transforming raw exported data into structured reports and dashboards while preserving the reporting requirements defined by the business team.

The proposed solution was developed around four Python-based reporting pipelines covering the main reporting activities, complemented by an interactive Graphical User Interface (GUI), a structured project architecture, historical snapshots, and automated execution mechanisms. The resulting system reduces the reporting preparation time from approximately one working day to a maximum of approximately 30 minutes for the complete reporting package.

This chapter presents the analysis of the existing process, the requirements identified from

the business team, the architecture of the proposed solution, the design of the four reporting pipelines, and the mechanisms implemented to facilitate their execution and maintenance.

3.2 Methodology

This project follows a **Design Science Research (DSR)** approach, the established methodology for Information Systems work whose primary objective is to design, build, and evaluate a technical artifact that addresses a real, practical problem [12]. This framing fits the present project closely: the outcome is not a theoretical model or a purely descriptive study, but a concrete software artifact – the four reporting pipelines and their shared interface – developed to solve an operational problem within the Channel Team and evaluated against real, previously manually-produced reports.

Peffers et al. structure DSR into six phases, each of which corresponds directly to a part of this report:

1. **Problem identification and motivation.** The existing manual reporting process was observed and documented to identify its concrete limitations – high preparation time, repetitive manual operations, risk of manual errors, and limited historical traceability. This is presented in Chapter 1 and in Section 3.3 of this chapter.

2. **Definition of the objectives of a solution.** The requirements for the automated solution were identified through discussions with the EBG Channel Team, combining a business perspective (what information the team needs) with a technical perspective (how that information can be obtained, transformed, and delivered automatically). This is presented in Section 3.4.

3. **Design and development.** The four reporting pipelines, the shared graphical user interface, and the supporting historical-snapshot and automation mechanisms were designed and implemented, as detailed in the remainder of this chapter and in Chapter 4.

4. **Demonstration.** The resulting artifact was used to generate real reporting outputs – dashboards, KPI cards, and comparison tables – as shown in Chapter 5.

5. **Evaluation.** The automated outputs were validated over approximately one month against previously manually-prepared reports, with discrepancies traced back to their cause and corrected before being compared again, as described in Chapter 5.

6. **Communication.** The results of this process are communicated through the present report.

Consistent with common practice in applied DSR projects, the design-and-development and evaluation phases were not executed as a single pass, but as a repeated cycle: each of the four pipelines was built, compared against the corresponding manual report, corrected, and re-compared, with feedback gathered directly from the Channel Team incorporated back into the artifact before it was considered complete. This is the reason the project is described, throughout this report, as having been developed *with* the business team rather than *for* it: at no point was the automated solution treated as finished until it had been observed in use and adjusted accordingly.

3.3 Analysis of the Existing Reporting Process

3.3.1 Current Reporting Workflow

Before the implementation of the proposed solution, the preparation of weekly reports was primarily performed manually. The process was handled by a member of the team and consisted of several sequential operations.

The general workflow included:

1. Extracting the required data from the relevant internal systems or exported files.
2. Opening and reviewing the source Excel files.
3. Cleaning and preparing the data manually.
4. Filtering the required information.
5. Using simple Excel functions and PivotTables to perform calculations and aggregations.
6. Cross-referencing and manually merging values calculated in one file (e.g. partner revenue) with reference information kept in a separate file (e.g. partner targets), since no single export already contained everything needed.
7. Manually transferring the resulting combined values into the existing Excel report template used by the team.
8. Manually reviewing the resulting figures for obvious inconsistencies or errors before the report was considered final.
9. Formatting and finalizing the report file for use in business reviews and partner meetings.

It is also worth noting that, prior to the proposed solution, the team did not work from dashboards: performance was read and interpreted directly from these generated tables, rather than through a dedicated visual summary layer. The introduction of a dashboard sheet is therefore one of the improvements introduced by the automated solution, discussed in Section 3.8.2.

Although the underlying data sources remained relatively stable, the volume of data made the manual treatment time-consuming. Consequently, some details that could theoretically be processed from the available source data were not systematically incorporated into the reporting process because processing them manually would have required additional effort.

The process could therefore be represented as a sequence of repetitive manual transformations:

$$\text{Raw Data} \rightarrow \text{Manual Cleaning} \rightarrow \text{Filtering} \rightarrow \text{Calculations} \rightarrow \text{Consolidation} \rightarrow \text{Reporting} \tag{3.1}$$

Here, *Consolidation* refers to the same cross-file merging step described above (item 6): bringing values calculated from different source files together into a single, coherent dataset before a report can be produced from it.

The repetitive nature of these operations indicated that a significant part of the process could be standardized and automated.

3.3.2 Limitations of the Existing Process

The analysis of the existing workflow highlighted several limitations.

High Preparation Time

The preparation of the reporting package required approximately one full working day. A considerable portion of this time was spent on repetitive operations rather than analytical activities.

Repetitive Manual Operations

Cleaning, filtering, matching, aggregating, and transferring information between files were repeatedly performed by the user. Such operations are suitable candidates for automation because they follow predefined rules.

Risk of Manual Errors

Repeated manual manipulation of large datasets increases the possibility of errors such as incorrect filtering, missed records, incorrect calculations, or inconsistent data transfers between files.

Limited Scalability

As the volume of source data increased, the manual process became increasingly difficult to maintain. Processing additional information required additional manual effort.

Dependence on Individual Knowledge

The reporting process depended significantly on the person responsible for its preparation. Knowledge of the required steps, calculations, filters, and source files was necessary to reproduce the reports correctly.

Limited Historical Traceability

The manual process did not provide a sufficiently structured mechanism for automatically preserving previous states of the processed data. This limited the ability to systematically compare successive reporting periods.

These limitations motivated the development of an automated reporting architecture capable of standardizing the recurring processing steps while maintaining the existing business reporting format.

3.4 Requirements Analysis

3.4.1 Requirement Gathering

The requirements of the proposed system were identified through discussions with the EBG Channel Team in order to understand the information required for operational monitoring and decision-making.

The business team provided the required reporting needs, KPIs, calculations, information to be included in the reports, and expected reporting formats. These requirements were subsequently translated into technical processing rules within the Python pipelines.

This approach ensured that the automation addressed actual business requirements rather than simply automating the existing manual actions without considering their purpose.

The resulting design therefore combines two perspectives:

- **Business perspective:** What information does the team need? This was provided directly by the Channel Team through the discussions described above.
- **Technical perspective:** How can this information be obtained, transformed, validated, and delivered automatically? This perspective corresponds to the analysis and design work

carried out during this project to translate the team's needs into a working automated solution, and is the subject of the remainder of this chapter.

This interaction between business requirements and technical implementation represents an important aspect of the Business Analytics dimension of the project.

3.4.2 Functional Requirements

The main functional requirements identified for the solution were:

- Import data from Excel and CSV source files.
- Clean and standardize raw data.
- Process large volumes of records.
- Integrate information from multiple source files.
- Apply business-defined calculations and rules.
- Calculate relevant KPIs.
- Compare current and historical information.
- Preserve historical snapshots.
- Preserve any value manually corrected or entered by the business team directly on a previous report (e.g. a partner's Status, or the assigned channel and Account Manager) when the same report is regenerated the following week, without requiring any change to the Python code to do so.
- Generate standardized Excel reports.
- Produce dashboards based on processed information.
- Provide a simple mechanism for launching report generation.
- Display execution information and errors when necessary.
- Generate outputs without requiring manual modification of Python code.

The solution was designed to support four principal reporting pipelines:

1. Weekly Partners Performance Report.

2. Channel Revenue Report.

3. eSales Report.

4. Certifications Report.

These four reports were prioritized over other, simpler reporting outputs because they were the most complex to prepare manually: each relies on a large number of business-specific formulas and classification rules, requires combining several source files rather than a single export, and processes a comparatively large volume of records. As a result, they represented both the greatest manual effort and the greatest risk of manual error, making them the strongest candidates for automation.

3.4.3 Non-Functional Requirements

In addition to functional requirements, several non-functional requirements influenced the design.

Efficiency

The system needed to significantly reduce the time required to prepare recurring reports.

Reliability

The same input data and business rules should produce consistent results across reporting periods.

Maintainability

The reporting scripts, input files, historical data, and generated outputs needed to be organized in a structured manner to facilitate maintenance.

Usability

Business users should be able to launch reports without interacting directly with the Python source code.

Traceability

The solution should maintain historical information to support comparisons between reporting periods.

Compatibility

The solution needed to preserve Excel as the principal reporting environment because the business team already relied on Excel-based reports and dashboards.

3.5 Proposed Solution

3.5.1 Solution Overview

To address the limitations identified in the existing process, an automated reporting system was developed using Python as the main processing technology and Microsoft Excel as the principal reporting and visualization environment.

The solution follows a lightweight ETL architecture:

$$\text{Source Data} \rightarrow \text{Extraction} \rightarrow \text{Transformation} \rightarrow \text{Validation} \rightarrow \text{Reporting} \tag{3.2}$$

The architecture was selected according to the practical requirements of the project. Since the organization already relies on Excel-based reporting and the objective was to improve the existing process without unnecessarily increasing its technological complexity, a lightweight Python-to-Excel architecture was considered appropriate.

The proposed system consists of:

- Raw data sources.
- Python ETL pipelines.
- Business rules and KPI calculations.
- Data validation mechanisms.
- Historical snapshot mechanisms.
- Excel reports and dashboards.
- A GUI for report execution.
- Batch execution through `.bat` files.

3.5.2 Overall System Architecture

The proposed architecture can be described through four main layers.

Input Layer

The input layer contains the raw files exported from the relevant Huawei systems and supporting reference files.

Depending on the reporting pipeline, these inputs may include:

- Raw Excel or CSV exports from Huawei's internal system.
- Historical datasets (prior-period revenue, certifications, contracts).
- Partner target and classification information.
- Reference and mapping files (e.g. university lookup, partner-level rules).
- Previous reporting snapshots, used for period-over-period comparison.

Processing Layer

The processing layer is implemented in Python. It performs the main ETL operations, including:

- Data extraction.
- Data cleaning.
- Data standardization.
- Data filtering.
- Data merging.
- Aggregation.
- KPI calculation.
- Period comparison.
- Validation.

Reporting Layer

The processed data is then used to generate the required Excel reports and dashboards. Existing company templates were preserved where appropriate and enhanced through modifications to accommodate the automated outputs.

User Layer

The final layer allows business users to execute the reporting pipelines. It includes the developed GUI and batch execution mechanism.

The overall architecture can therefore be summarized through four main layers, as illustrated in Figure 3.1: input, processing, reporting, and user layers, each responsible for a distinct stage of the pipeline from raw data extraction to business use.



Figure 3.1: Overall architecture of the automated reporting system
Source: Author

3.6 Design of the Automated Reporting Pipelines

3.6.1 Common Pipeline Architecture

Although each report has its own specific business logic and source files, the four pipelines follow a common conceptual structure: each supplies its own distinct set of input files to the same shared sequence of processing stages, and produces its own distinct Excel report. Figure 3.2 illustrates this structure across all four pipelines together.

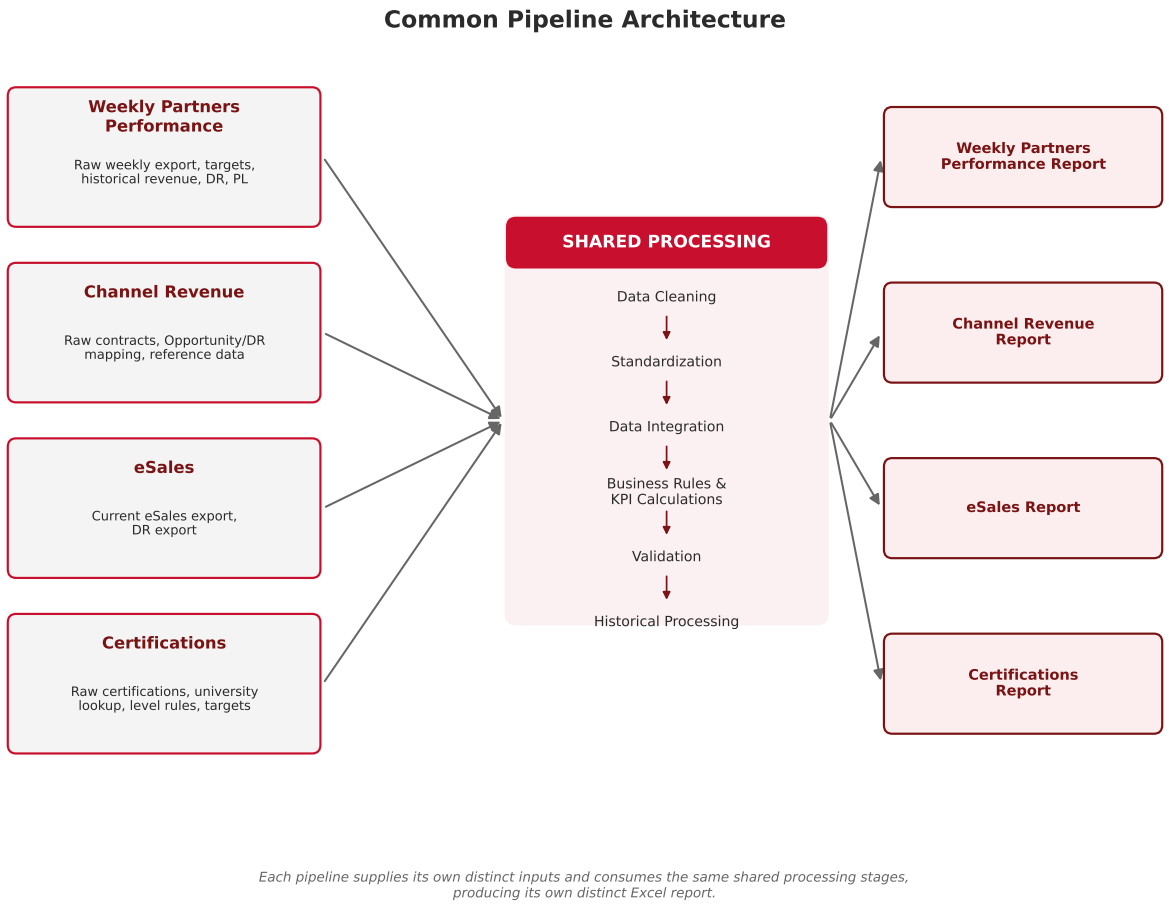


Figure 3.2: Common architecture shared by the four reporting pipelines
Source: Author

Each pipeline is implemented as an independent Python module (`clean_weekly_report.py`, `clean_channel_revenue_report.py`, `clean_esales_report.py`, `clean_certifications.py`). This means that a modification to one reporting process does not necessarily require modifications to the others.

The project structure developed for the solution separates the different reporting domains into dedicated, self-contained folders, each holding its own processing script, its own raw and reference inputs, and its own historical snapshot subfolder:

- `weekly`
- `contracts`
- `esales`
- `certification`

The concrete file-level layout of this structure, along with the folder-by-folder inventory of scripts and input files, is presented in Chapter 4 (Implementation).

3.6.2 Weekly Partners Performance Pipeline

The Weekly Partners Performance pipeline is responsible for preparing the weekly partner performance reporting.

The pipeline uses several input sources:

- **Raw weekly partner data:** the current week's export of per-partner order and revenue figures, as produced by Huawei's internal system.
- **Partner targets and revenue:** a reference file listing each partner's assigned yearly target, used to compute target achievement percentages.
- **Historical revenue:** the corresponding totals from the prior year, used to compute Year-over-Year growth.
- **Raw DR export:** a separate Deal Registration export, structured differently from the main partner data and used specifically to build the NC Achievement table for a defined set of channel partners.
- **Historical PL:** historical Product-Line-level revenue data, used where the reporting logic requires a breakdown by product line rather than only a partner-level total.

An optional Channel Revenue report output can also be supplied to support specific Channel/Account-Manager splits for select rows.

The pipeline automates the preparation of information that previously required manual filtering, aggregation, and consolidation, including year-to-date order and revenue totals, target achievement percentages, and Month-over-Month and Year-over-Year growth for every partner.

Historical information, preserved between runs through a dedicated snapshot mechanism, is incorporated into the process to support comparisons between reporting periods. The design of this snapshot mechanism, along with the pipeline's data-quality safeguards (such as consolidating partners that appear on more than one row of the raw export), is detailed in Chapter 4.

3.6.3 Channel Revenue Pipeline

The Channel Revenue pipeline processes channel revenue information using several complementary sources.

The main inputs include the raw Channel Revenue export and an Opportunity/DR mapping file. Additional history, reference data, and Tier 1 purchase-order reference information can be provided when required.

A central design decision in this pipeline is the automatic classification of every raw contract row into a Partner Type (Carrier, DEMO, STOCK, NC, Non-NC, or Direct) using an explicit, editable set of keyword and Tier-1/Tier-2 matching rules, rather than requiring the business user to assign this classification by hand for every contract. The use of dedicated mapping and reference files similarly allows the pipeline to integrate information that is not necessarily contained within a single raw export, an important requirement, since business information (such as the responsible Account Manager or channel) frequently needs to be associated with additional organizational or opportunity-related attributes before the final report can be produced.

Because several of the report's fields (Status, channel, Partner AM) are expected to remain partly business-maintained rather than fully derived from the raw data, the pipeline's design explicitly preserves these manually-entered values across successive runs rather than overwriting them each time the report is regenerated.

3.6.4 eSales Pipeline

The eSales pipeline automates the processing of eSales information.

Its main input files are the current-week eSales export and the corresponding DR export.

The objective is to reduce the manual work required to prepare the eSales reporting output while maintaining the structure expected by the business users. Part of this objective includes preserving visual flags already used by the team in the raw export, specifically, cell highlighting used to mark opportunities that need to be deleted, so that information the business team already relies on is not lost when the report is regenerated automatically. The pipeline also produces a dedicated *Ongoing Pipe* view scoped to the Channel Team's named partners, reflecting their existing tiering and commercial status.

3.6.5 Certifications Pipeline

The Certifications pipeline automates the preparation of certification-related reporting.

The dedicated certification project folder contains the certification processing script together

with supporting files such as:

- Raw certification data.
- Previous certification snapshots.
- Partner domain-level snapshots.
- Partner-level rules.
- Partner targets.
- University lookup information.
- Generated certification reports.

The pipeline therefore combines current certification information with supporting reference and historical information to generate the required reporting output.

An important design requirement for this pipeline is that the raw certification data mixes genuine partner certifications with university-issued certificates, which must be excluded from partner-level reporting. Rather than hard-coding this distinction, the design routes it through an editable university lookup table, and, consistent with the project's general principle of flagging rather than guessing, any partner name not found in that lookup is routed to a dedicated review sheet instead of being silently included or excluded. The pipeline was also designed to derive an overall and per-domain partner level (Basic/Advanced/Master) from a separate, editable rules table, and to compare each partner's current level against its target level, so that certification gaps are surfaced directly in the report rather than requiring a separate manual analysis.

The use of explicit rule and reference files also separates business configuration from the processing code, making the reporting process easier to update when requirements evolve.

3.7 Data Transformation and Processing

3.7.1 Data Extraction

The first stage of each pipeline consists of retrieving the necessary information from the source files.

The solution primarily works with Excel and CSV files. The GUI allows users to select the appropriate input files through file-browsing controls rather than requiring them to manually edit file paths in the Python scripts.

This reduces configuration errors and makes the reporting process accessible to users who do not need to interact directly with the source code.

3.7.2 Data Cleaning and Standardization

Raw exported data is not always directly suitable for analysis. Consequently, the Python pipelines perform cleaning and standardization operations before calculations are performed.

These operations form an essential part of the ETL process because they ensure that subsequent calculations operate on a consistent dataset.

Depending on the reporting pipeline, the transformation stage may include operations such as:

- Removing unnecessary records or columns.
- Standardizing values.
- Handling missing information.
- Converting data into appropriate formats.
- Filtering relevant records.
- Preparing fields required for calculations.
- Aligning information originating from different sources.

The main advantage of implementing these operations programmatically is that the same transformation rules can be applied consistently every time the report is generated.

3.7.3 Data Integration

Several reporting pipelines require information from multiple files.

Instead of manually copying information from one Excel file to another, the Python scripts automate the integration of these datasets.

This is particularly important for:

- Partner-level reporting.
- Opportunity and DR mapping.
- Historical comparisons.
- Target integration.
- Reference information.
- Certification classification.

The integration stage transforms multiple independent exports into datasets suitable for reporting.

Conceptually:

$$D_{final} = D_1 \cup D_2 \cup \cdots \cup D_n \tag{3.3}$$

where each D_i represents a relevant source or reference dataset.

3.7.4 KPI and Business Rule Processing

The KPIs and business rules were not arbitrarily defined during the technical implementation. They were identified through discussions with the Huawei business team based on the information required for operational monitoring.

The technical role of the Python pipelines was therefore to translate these requirements into repeatable processing rules.

This distinction is important because the project combines **business requirements analysis** with **technical implementation**.

The resulting process can be represented as:

Business Requirements → Required KPIs and Information → Technical Processing Rules → Python Implementation → Automated Reporting

This approach ensures that automation remains aligned with the actual reporting and decision-support needs of the team.

3.7.5 Historical Snapshots and Comparisons

One of the main architectural improvements introduced by the project is the implementation of historical snapshot mechanisms.

The project contains dedicated snapshot and history structures for different reporting processes, including:

- Weekly comparison snapshots.
- eSales pipeline snapshots.
- Contract snapshots.
- Previous certification snapshots.

The purpose of these mechanisms is to preserve relevant historical states of the processed data.

Instead of treating every reporting period as an isolated operation, the system can retain previous information and use it for comparison. Each pipeline searches its snapshot folder for the closest matching prior state rather than requiring an exact date match, and reports a metric as unavailable rather than computing a misleading value when no sufficiently old snapshot yet exists — for example, immediately after the system is first deployed.

This supports reporting requirements such as:

$$\Delta = Value_{current} - Value_{previous} \tag{3.4}$$

and, where relevant:

$$Growth\ Rate = \frac{Value_{current} - Value_{previous}}{Value_{previous}} \times 100 \tag{3.5}$$

These comparisons are also made visually explicit in the generated reports rather than left as numbers the reader has to interpret unaided: cells whose value increased or decreased relative to the previous snapshot are directly color-coded in place (Section 3.8.2), so that the direction of change is visible at a glance on the same figure being read.

The snapshot mechanism therefore improves both traceability and analytical capabilities.

3.8 Reporting and Dashboard Design

3.8.1 Excel Reporting Layer

Microsoft Excel was retained as the principal reporting environment because it is already familiar to the business users and corresponds to the organization's existing reporting practices.

This choice was also shaped by a practical constraint: Huawei's internal information systems are proprietary and access-restricted, which limits the Channel Team's ability to integrate directly with these systems or to deploy additional third-party BI or dashboarding tools on top of them. Excel was therefore not only the environment business users were already comfortable with, but effectively one of the few reporting tools realistically available given these access constraints.

Rather than replacing the existing reporting format entirely, the project preserves the required Excel templates and enhances them where necessary.

The role of Python is therefore primarily to automate the preparation of the underlying information, while Excel remains the final user-facing reporting environment.

This separation provides a practical architecture:

Python → Data Preparation and Automation → Excel → Reports and Dashboards → Business Users

3.8.2 Dashboard Development

The dashboards were developed as part of the project to improve the visualization and interpretation of the processed information.

The dashboards consolidate relevant indicators into a more accessible format and allow users to monitor performance without manually reproducing calculations from raw datasets. Beyond a summary dashboard sheet, the reporting workbooks use in-place visual cues, conditional color-scale formatting on achievement and growth figures, and direct highlighting of values that increased or decreased since the previous period, so that performance signals are visible on the same figure the user is already reading, rather than requiring a separate chart to be cross-referenced.

The design follows the principle that reporting should not only provide data but should facilitate rapid interpretation.

The dashboard layer therefore acts as the bridge between processed data and business decision-making.

3.8.3 Preservation and Enhancement of Existing Templates

The Excel templates used by the team were not completely redesigned from scratch.

Instead, the existing reporting structures required by the business were preserved and enhanced where necessary. Modifications were introduced to accommodate the automated data outputs and improve the usability of the resulting reports.

This approach reduced organizational disruption because users could continue working with a familiar reporting environment while benefiting from automated data preparation.

3.9 Automation Architecture

3.9.1 Python-Based Automation

Python constitutes the core processing technology of the proposed system.

The four reporting pipelines were independently developed from scratch.

This represents one of the main technical contributions of the project.

Each pipeline encapsulates the processing logic associated with a specific reporting domain, making it possible to execute, maintain, and improve each reporting process independently.

3.9.2 Graphical User Interface

To further simplify the execution of the reporting system, a dedicated Graphical User Interface (GUI) was developed using Python's Tkinter framework.

The GUI provides a user-facing layer between the business user and the Python pipelines, and was designed to be reused unmodified across all four reporting domains: rather than building a separate interface per report, a single GUI module is placed in each domain folder and adapts itself to whichever script is actually present alongside it.

At startup, the application automatically detects which report scripts are available in the current folder and displays only the corresponding reporting options. This design prevents users from selecting reports whose corresponding scripts are not present.

The interface allows users to:

- Select a report.
- Browse for required input files.
- Identify optional files.
- Launch report generation.
- Monitor execution logs.
- Receive success or failure notifications.
- Open the generated report's output folder.

The GUI was therefore designed to minimize the technical knowledge required from the final user. Its concrete implementation is presented in Chapter 4.

3.9.3 Asynchronous Report Execution

The GUI incorporates a separate execution thread for report generation.

When the user selects the *Build Report* option, the interface launches the corresponding reporting module in a background thread instead of blocking the graphical interface.

This improves usability because the application remains responsive while the report is being generated.

The GUI also captures standard output and error output from the Python process and displays them in an integrated log area. This allows users to monitor the execution process and identify potential problems.

3.9.4 Error Handling and Execution Feedback

The reporting interface provides explicit execution feedback.

When a report is successfully generated, the interface displays a completion status and provides the option to open the output folder. When an error occurs, the system displays an error notification and preserves the detailed execution information in the log.

This mechanism improves transparency and facilitates troubleshooting.

3.9.5 Batch Execution

In addition to the GUI, batch execution was implemented using Windows `.bat` files.

The batch mechanism provides a simple way to launch the reporting application without requiring the user to open a terminal and manually execute Python commands.

The execution mechanism checks whether the `uv` Python environment manager is available. If it is available, the reporting application is executed using `uv run python`; otherwise, the system falls back to the standard Python interpreter.

This provides flexibility across environments while maintaining a simple user workflow.

The batch execution layer can therefore be viewed as an additional abstraction:

Business User → Batch Launcher → GUI → Python Pipeline → Excel Report

3.10 Project Organization and Maintainability

3.10.1 Structured Folder Architecture

A dedicated folder structure was designed for the reporting system.

The main project directory separates each reporting domain into its own folder. This organization allows scripts, raw inputs, historical data, snapshots, and generated outputs to remain associated with their respective reporting processes.

The structure also facilitates future maintenance because changes to one reporting domain can be isolated from the others.

3.10.2 Separation of Processing and Configuration

The architecture also separates processing scripts from configuration and supporting files.

For example, partner targets, mappings, reference information, and historical datasets are maintained separately from the Python processing logic.

This design reduces the need to modify source code when certain business inputs change.

3.10.3 Reusability

The pipelines were designed for recurring use.

Instead of creating a report once, the system allows the same processing workflow to be executed repeatedly using new exports.

This changes the reporting process from a sequence of manually repeated operations into a reusable reporting workflow:

$$\text{New Data} \rightarrow \text{Run Pipeline} \rightarrow \text{Updated Report} \tag{3.6}$$

Consequently, the technical effort is concentrated on maintaining the transformation rules rather than repeatedly reproducing the same manual operations.

3.11 Summary of the Proposed Solution

The proposed reporting system combines several complementary components, summarized in Table 3.1.

Table 3.1: Main components of the proposed reporting solution

Component	**Purpose**
Python pipelines	Automate data extraction, transformation, integration, and calculations.
Weekly pipeline	Automate partner performance reporting.
Channel Revenue pipeline	Automate channel revenue reporting.
eSales pipeline	Automate eSales reporting.
Certifications pipeline	Automate certification reporting.
Historical snapshots	Preserve previous reporting states.
Excel reports	Provide business-oriented reporting outputs.
Excel dashboards	Facilitate performance monitoring and visualization.
GUI	Simplify report execution.
`.bat` launcher	Provide simple application startup.
Structured folders	Improve organization and maintainability.

Source: Author

The solution was entirely developed within the scope of the project, with the four Python pipelines, GUI, folder structure, snapshot mechanisms, dashboards, and batch execution mechanism created specifically to address the identified reporting requirements.

3.12 Conclusion

This chapter presented the analysis and design of the automated reporting solution developed for Huawei's Enterprise Business Group.

The analysis of the existing reporting process demonstrated that the preparation of recurring reports involved a significant amount of repetitive manual work, including data cleaning, filtering, calculation, consolidation, and report updating. The process required approximately one working day and became increasingly difficult to maintain as data volumes increased.

To address these limitations, a lightweight ETL-based architecture was designed and implemented using Python and Excel. Four independent Python pipelines were developed from scratch to automate the main reporting activities: Weekly Partners Performance, Channel Revenue, eSales, and Certifications.

The solution was further enhanced through the implementation of historical snapshots, a structured project architecture, automated batch execution, and a dedicated Graphical User Interface. The GUI provides a simplified interface for selecting input files, launching reports, monitoring execution, and accessing generated outputs.

Excel was deliberately retained as the principal reporting and visualization environment. Existing business templates were preserved and enhanced rather than completely replaced, ensuring continuity for the users.

Overall, the architecture transforms the original manual workflow into a reusable automated reporting process. The next chapter moves from design to implementation, providing a detailed examination of the Python pipelines, data transformations, historical mechanisms, GUI, dashboards, and generated reporting outputs.

Chapter 4

Implementation

4.1 Introduction

The previous chapter presented the analysis of the existing reporting process and the architecture of the proposed solution, structured around four independent Python pipelines, a shared Graphical User Interface, and Excel as the principal reporting environment.

This chapter presents the concrete implementation of that architecture: the project's folder organization, the internal structure of each of the four reporting scripts, the shared conventions they follow (styling, snapshot management, manual-value preservation), and the design of the GUI that ties them together. Code excerpts are used throughout to illustrate implementation choices that would otherwise remain abstract at the design level.

4.2 Project Structure

The solution is organized as a single project directory, with one dedicated subfolder per reporting domain: `weekly`, `contracts`, `esales`, and `certification`. Each subfolder is self-contained: it holds its own processing script, its own raw and reference input files, its own historical/snapshot subfolder, its own generated report, and a copy of the shared `gui_reports.py` launcher and `run_reports.bat` file.

```
PFE_Project/
|-- certification/
|   |-- clean_certifications.py
|   |-- gui_reports.py
|   |-- run_reports.bat
|   |-- raw_certifications.xlsx
|   |-- university_lookup.csv
|   |-- partner_level_rules.csv
|   |-- partner_targets.csv
|   |-- certifications_snapshot_previous.csv
|   |-- partner_domain_levels_snapshot_previous.csv
|   `-- certifications_report.xlsx
|-- contracts/
|   |-- clean_channel_revenue_report.py
|   |-- gui_reports.py
|   |-- run_reports.bat
|   |-- raw_channel_revenue.xlsx
|   |-- opportunity_dr_mapping.xlsx
|   |-- reference_data.xlsx
|   |-- tier1_po_reference.xlsx
|   |-- history_data.xlsx
|   |-- contract_snapshots/
|   `-- CHANNEL_REVENUE_REPORT.xlsx
|-- esales/
|   |-- clean_esales_report.py
|   |-- gui_reports.py
|   |-- run_reports.bat
|   |-- esales_current.xlsx
|   |-- esales_previous.xlsx
|   |-- dr_export.xlsx
|   |-- esales_pipe_snapshots/
|   `-- ESALES_REPORT.xlsx
`-- weekly/
    |-- clean_weekly_report.py
    |-- gui_reports.py
    |-- run_reports.bat
    |-- raw_weekly_partner_data.xlsx
    |-- raw_dr_export.xlsx
    |-- partner_targets_revenue.csv
    |-- historical_revenue.csv
    |-- historical_pl.xlsx
    |-- weekly_comparison_snapshots/
    `-- WEEKLY_PARTNERS_PERFORMANCE_REPORT.xlsx
```

Figure 4.1: Project folder structure – one self-contained subfolder per reporting domain
Source: Author

This organization reflects the ”separation of processing and configuration” principle introduced in Chapter 3: business inputs (partner targets, reference tables, university lookups, partner-level rules) are kept as standalone CSV/Excel files next to the script that consumes them, rather than

hard-coded inside the Python logic. Each domain folder also contains a dedicated snapshot subfolder (`weekly_comparison_snapshots`, `contract_snapshots`, `esales_pipe_snapshots`) used to preserve historical states of the processed data between runs, and a `.pycache__` folder generated automatically by the Python interpreter.

Because each folder is self-contained, a report can be executed independently of the others: running the Weekly pipeline does not require the Channel Revenue or Certifications folders to be present, which keeps the four reporting domains loosely coupled and lets each one evolve on its own schedule.

4.3 Common Implementation Conventions

Although each of the four pipelines addresses a different reporting domain, they were implemented following shared conventions that make the codebase more predictable to maintain.

4.3.1 Configuration Constants

Every script exposes its file paths, business-rule parameters, and styling constants as module-level constants near the top of the file, rather than embedding them inside function bodies. For example, the weekly pipeline defines:

```
RAW_FILE_PATH = "raw_weekly_partner_data.xlsx"
TARGET_FILE_PATH = "partner_targets_revenue.csv"
HISTORICAL_FILE_PATH = "historical_revenue.csv"
SNAPSHOT_DIR = "weekly_snapshots"
OUTPUT_PATH = "WEEKLY_PARTNERS_PERFORMANCE_REPORT.xlsx"
CHANNEL_TARGET_SOURCE_COL = "TargetRevenue_YTD_MUSD"
```

This convention serves two purposes. First, it allows the GUI (Section 4.8) to override these values at runtime with the file paths selected by the business user, without modifying the underlying script. Second, it isolates business-rule decisions – such as which partners are included in a given total, or which column is treated as the authoritative revenue target – as named, documented constants rather than values embedded in calculation logic, which makes them easier to review and adjust as requirements evolve.

4.3.2 Robust File Reading

Because the source files are exported by business users from different systems, the scripts do not assume a single fixed encoding. A shared helper is used across the pipelines to fall back to a

Windows encoding when UTF-8 decoding fails:

```
def read_csv_safe(path, **kwargs):
    try:
        return pd.read_csv(path, encoding="utf-8", **kwargs)
    except UnicodeDecodeError:
        return pd.read_csv(path, encoding="cp1252", **kwargs)
```

Similarly, several pipelines accept either an Excel or a CSV file for the same logical input and dispatch to the appropriate `pandas` reader based on the file extension, which reduces the number of format-specific errors business users can trigger simply by exporting a file differently than expected.

4.3.3 Manual-Value Preservation

A recurring requirement identified during the requirements analysis (Chapter 3) was that some fields in the generated reports are not fully derivable from the raw data and are expected to be corrected or completed by hand – for example, a partner's commercial *Status*, or the *channel*, *Partner AM*, and *Huawei OO* fields in the Channel Revenue report. Because each pipeline regenerates its output file from scratch on every run, a naive implementation would silently erase these manual corrections the next time the report is built.

To address this, the Weekly and Channel Revenue pipelines both read their own *previous* output file before writing the new one, and carry forward any manually-entered value for a defined set of fields when the automated logic does not produce one, or when the previous value was itself a manual override:

```
MANUAL_CARRY_FORWARD_FIELDS = ["channel", "Partner AM", "Huawei OO",
                               "Tier 1 PO ref", "T1 PO Amount Euro"]


def load_previous_manual_values(path):
    """Reads the 'Channel Revenue Report' sheet from the PREVIOUS run's
    output file (if it exists) and captures whatever's sitting in the
    manually-maintained fields, keyed by Contract."""
    ...
```

Records are matched between runs using a stable business key (the contract number for the Channel Revenue report, the normalized partner name for the Weekly report) rather than row position, so the carry-forward logic remains correct even if the row order changes between runs.

4.3.4 Historical Snapshots and Period Comparison

All four pipelines write a dated snapshot of their processed output to a dedicated subfolder at the end of every run, and read the closest-matching prior snapshot at the start of the next run to compute period-over-period changes (Month-over-Month growth, week-over-week revenue deltas, newly detected contracts or certificates). The Weekly pipeline's snapshot lookup illustrates this pattern: rather than requiring an exact date match, it searches the snapshot folder for the file whose date is closest to a target number of days in the past, while enforcing a minimum age so that a snapshot from earlier the same week cannot be mistaken for a month-old one:

```
def find_snapshot_near(days_ago, min_gap_days=20):
    folder = Path(SNAPSHOT_DIR)
    candidates = []
    for f in folder.glob("snapshot_*.csv"):
        date_str = f.stem.replace("snapshot_", "")
        d = datetime.strptime(date_str, "%Y-%m-%d")
        gap = (TODAY - d).days
        if gap >= min_gap_days:
            candidates.append((abs(gap - days_ago), f))
    candidates.sort(key=lambda x: x[0])
    return read_csv_safe(candidates[0][1]) if candidates else None
```

When no sufficiently old snapshot exists yet typically on the first few runs after deployment the affected metric is reported as *"Not enough history yet"* rather than a computed value, so that the report never silently presents a comparison that is not actually meaningful.

4.3.5 Report Styling

Each pipeline writes its output using `openpyxl` rather than relying on `pandas`' default Excel export, in order to fully control formatting: a Huawei-red header row, alternating row banding, frozen header panes, autofilter, and column widths fitted to content. Color-scale and data-bar conditional formatting are applied to Achievement and Growth columns so that over/under-performance is visible at a glance without reading exact figures:

```
rule = ColorScaleRule(
    start_type="num", start_value=0, start_color="F4B6B6",
    mid_type="num", mid_value=100, mid_color="FCE4A6",
```

```
    end_type="num", end_value=150, end_color="B7DDB0",
)
ws.conditional_formatting.add(rng, rule)
```

Cells whose value increased or decreased relative to the previous snapshot are additionally tinted green or red directly in place, rather than through a separate delta column, so that the change is visible on the same figure the user is already reading.

4.4 Weekly Partners Performance Pipeline

The Weekly pipeline (`clean_weekly_report.py`) merges the current-week raw partner export with two manually-maintained reference files (partner targets and revenue history) keyed on Partner Code, then computes, for every partner: year-to-date orders and revenue, target achievement percentage, Month-over-Month growth (via the snapshot mechanism above), and Year-over-Year growth against the prior year's historical totals.

```
order_achievement = round((ytd_booking / 1_000_000) / target_booking * 100, 1)
...
if mom_lookup is not None and code in mom_lookup.index:
    prev = mom_lookup.loc[code]
    mom_order = round((ytd_booking - prev[YTD_BOOKING_COL]) / 1000, 1)
else:
    mom_order = "Not enough history yet"
```

Before merging, an explicit consolidation step sums the numeric columns of any partner appearing on more than one row of the same source file, rather than silently dropping the duplicate – a data-quality issue observed in the raw exports during the analysis phase:

```
def sum_duplicates(df, label):
    dupes = df[df.duplicated(PARTNER_CODE_COL, keep=False)]
    if dupes.empty:
        return df
    print(f"NOTE: {label} has partner(s) appearing in multiple rows -- "
          f"summing their numeric columns into one row.")
    ...
```

Beyond the main partner table, the pipeline also builds an *NC Achievement* table restricted to seven specific channel partners, computed from a separate raw Deal Registration (DR) export whose column headers are bilingual (Chinese/English). Partner classification in this table

combines a case-insensitive name match with an optional BG-code match, so that a partner can still be correctly attributed even when its name is misspelled or abbreviated differently across exports:

```
NC_PARTNER_BG_CODES = {
    "Partner A": ["XXXXXXXXXX"],
}
```

The output workbook contains the main performance table, a KPI-card dashboard sheet, the NC Achievement table, and a dedicated *Weekly Changes* section summarizing order and revenue deltas since the previous snapshot.

4.5 Channel Revenue Pipeline

The Channel Revenue pipeline (`clean_channel_revenue_report.py`) is the most rule-intensive of the four: it classifies every raw contract row into a Partner Type (Carrier, DEMO, STOCK, NC, Non-NC, or Direct) using an ordered set of keyword and Tier-1/Tier-2 matching rules defined as module-level constants, so that the classification logic can be audited and adjusted without touching the surrounding code:

```
CARRIER_KEYWORDS = ["carrier operator 1", "carrier operator 2", "carrier operator 3"]
NC_TIER1_KEYWORDS = ["partner a", "partner a full name"]
NC_TIER2_KEYWORDS = ["partner b", "partner c"]
NONNC_TIER1_KEYWORDS = ["partner d", "partner d alt spelling", "partner e"]
```

A word-boundary matching helper is used throughout the classification logic to avoid short entity codes accidentally matching inside unrelated words:

```
def contains_word(haystack, word):
    return re.search(r"\b" + re.escape(word) + r"\b", haystack,
                     re.IGNORECASE) is not None
```

The pipeline also implements the manual-value carry-forward mechanism described above for the *channel*, *Partner AM*, *Huawei OO*, and Tier 1 PO fields, and detects new or changed contracts between runs by comparing against the closest previous snapshot in `contract_snapshots/`, reporting the number of new and modified contracts directly in the console log at the end of each run.

4.6 eSales Pipeline

The eSales pipeline (`clean_esales_report.py`) processes the current-week eSales export together with a Deal Registration export, and reconstructs the fill color originally applied to certain rows in the raw Excel export, used by the business team to flag opportunities without a matching DR entry, since this information is not exposed directly through `pandas` and has to be read from the underlying `openpyxl` cell styles:

```
def resolve_fill_hex(fgColor):
    """Best-effort: returns a 6-char hex string for a cell's fill color,
    whichever way Excel actually stored it (raw RGB, theme+tint, or an
    indexed palette color)."""
    ...
```

The pipeline also builds an *Ongoing Pipe* table restricted to ten named channel partners, matched either on their Tier 1 or Tier 2 column depending on the partner (distributors are matched on Tier 1, smaller resellers on Tier 2), and pre-assigns each partner a fixed commercial status (Diamond, Distributor, Gold, Silver, Authorized) used for display rather than recomputed from the data:

```
PIPE_PARTNER_STATUS = {
    "Partner A": "Diamond",
    "Partner D": "Distributor",
    ...
}
```

4.7 Certifications Pipeline

The Certifications pipeline (`clean_certifications.py`) differs from the other three in that its raw input mixes genuine partner certifications with university-issued certificates that must be excluded from partner-level reporting. An editable lookup table (`university_lookup.csv`) is used to filter these out, with any partner name not found in the lookup flagged into a separate *needs review* sheet rather than being silently included or excluded:

```
print("Filtering universities...")
clean_df, needs_review_df = filter_universities(raw_df)
print(f"  {len(clean_df)} rows kept, "
      f"{len(needs_review_df)} unknown partners flagged for review")
```

From the filtered data, the pipeline computes, per partner: certificate counts by level, certificates expiring within the current year (sorted by urgency, with a suggested renewal window), an overall and per-domain partner level (Basic/Advanced/Master) derived from a separate, editable rules table (`partner_level_rules.csv`), and a gap analysis comparing each partner's current level against the target level defined for them in `partner_targets.csv`. As with the other pipelines, a snapshot of the current certificate list is saved at the end of every run so that the next run can identify newly obtained certificates.

The output workbook therefore contains one sheet per insight (certificate counts, expiring certificates, partner levels, per-domain levels, gap analysis, new certificates) in addition to a summary dashboard, all styled with the same shared conventions used across the other three reports.

4.8 Graphical User Interface

4.8.1 Design Goal

The four pipelines were deliberately implemented as plain Python scripts driven entirely by module-level constants, with no command-line arguments or interactive prompts of their own. This design choice makes them straightforward to override programmatically – which is exactly what the GUI (`gui_reports.py`) does: rather than duplicating each pipeline's business logic, it imports the corresponding module at runtime and overwrites its configuration constants with the file paths selected by the user, before calling the module's `main()` function.

4.8.2 Report Auto-Detection

Because a copy of `gui_reports.py` is placed in every domain folder alongside its matching `clean_*.py` script, the same launcher file is reused across all four folders. At startup, it scans its own directory and only offers the reports whose script is actually present next to it:

```
def _detect_available_reports():
    available = {}
    for name, spec in REPORT_CATALOG.items():
        script_path = os.path.join(SCRIPT_DIR, spec["module"] + ".py")
        if os.path.exists(script_path):
            available[name] = spec
    return available
```

A copy placed in the `weekly` folder therefore only ever shows the Weekly Partners Performance report, while a copy in `contracts` shows the Channel Revenue report eliminating the possibility of a user selecting a report whose script and input files are not actually in that folder. As shown in Figure 4.2, the interface then dynamically builds one labeled, browsable input field per required or optional file for the selected report, based on the `REPORT_CATALOG` definition.

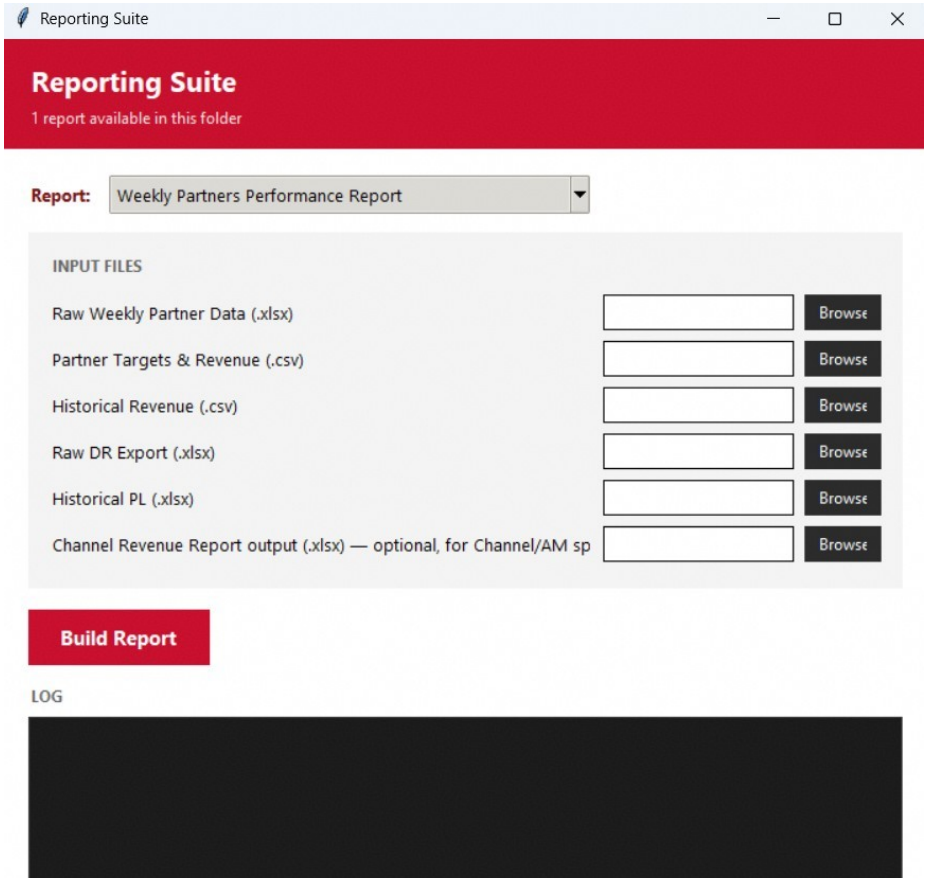


Figure 4.2: Reporting Suite GUI – Weekly Partners Performance Report screen
Source: Author

4.8.3 Dynamic Configuration Injection

When the user clicks *Build Report*, the GUI validates that every required (non-optional) field has been filled, then dynamically re-imports the target module and overwrites its configuration constants such as `RAW_FILE_PATH` or `TARGET_FILE_PATH` with the paths selected through the file browser, before invoking its `main()` function:

```
mod = importlib.import_module(module_name)
for config_name, label, optional in report["fields"]:
    val = self.field_vars[config_name].get().strip()
    if val:
        setattr(mod, config_name, val)
```

```
...
mod.main()
```

This approach means the GUI requires no changes to the underlying reporting scripts and no duplicated business logic, adding a fifth report to the suite in the future only requires adding one entry to `REPORT_CATALOG` naming its module and its expected input fields.

4.8.4 Asynchronous Execution and Logging

Report generation is run in a background thread rather than on the GUI's main thread, so the interface remains responsive while a pipeline executes. The pipeline's standard output and error streams are redirected into a queue and displayed in a live log panel, giving the business user visibility into the same progress messages (*"Cleaning...", "Building dashboard...", "Writing report..."*) that would otherwise only appear in a terminal:

```
class _Tee(io.StringIO):
    def write(_self, s):
        self.log_queue.put(s)
        return len(s)


tee = _Tee()
with redirect_stdout(tee), redirect_stderr(tee):
    mod.main()
```

On completion, the interface reports success or failure explicitly: a successful run offers to open the output folder directly, while a failed run keeps the full traceback available in the log panel rather than only showing a generic error message, which facilitates troubleshooting without requiring the business user to re-run the script from a terminal.

4.8.5 Batch Launcher

A `run_reports.bat` file is placed alongside each GUI copy so that the business user can start the application with a double-click, without opening a terminal or typing a Python command. The batch script checks whether the `uv` Python environment manager is available and, if so, launches the GUI through `uv run python`; otherwise it falls back to the system's default Python interpreter, which keeps the launch mechanism working across machines that may or may not have `uv` installed.

4.9 Conclusion

This chapter presented the concrete implementation of the four reporting pipelines and the GUI that operates them, grounded in the actual project structure and code. The four scripts share a common set of conventions – configuration constants, resilient file reading, manual-value preservation, historical snapshots, and consistent Excel styling – while each implements the specific business rules of its own reporting domain, from the Partner-Type classification logic in the Channel Revenue pipeline to the certificate-level and gap-analysis calculations in the Certifications pipeline. The GUI ties the four pipelines together without duplicating their logic, by dynamically configuring and invoking each module based on the files the business user selects.

The next chapter evaluates the results of this implementation, comparing the automated outputs against the previous manual process and discussing the solution's current limitations and possible extensions.

Chapter 5

Results and Evaluation

5.1 Introduction

The previous chapter presented the implementation of the automated reporting system: four independent Python pipelines covering Weekly Partners Performance, Channel Revenue, eSales, and Certifications reporting, tied together by a shared Graphical User Interface.

This chapter evaluates the resulting solution against the objectives set out in Chapter 3 (Analysis and Design): reducing preparation time, improving reliability and consistency, and preserving the reporting formats already used by the Channel Team. It first describes the approach used to validate the automated outputs against the previous manual process, then presents the results obtained in terms of time efficiency, reliability, and business use, before discussing the solution's current limitations and directions for future work.

5.2 Evaluation Methodology

Because the objective of the project was to automate an existing, business-validated reporting process rather than to introduce new indicators, the primary evaluation criterion was **consistency with the previous manually-produced reports**: for a given set of source files, the automated pipeline had to reproduce the same figures the team would have obtained manually, before any new automation-specific benefit (speed, historical comparison, dashboards) could be considered valid.

This was verified iteratively during development, as already described in Chapter 1: each pipeline's output was compared against a corresponding manually-prepared report for the same reporting period, discrepancies were traced back to their cause (a missed business rule, a data-quality issue in the source file, or a genuine calculation difference), and the pipeline was corrected before being compared again. This cycle was repeated for each of the four reporting domains

until the automated and manual outputs matched for the periods used as reference.

This validation process extended over approximately one month following the initial implementation of each pipeline, during which the automated output was repeatedly compared against manually-prepared reports, adjusted, and re-tested as issues were identified. Over this period, numerous discrepancies were identified and resolved ranging from data-quality issues in the source exports (such as partners appearing on multiple rows of a raw export) to gaps in the automated business rules until the automated and manual outputs matched consistently across the periods used as reference.

5.3 Time Efficiency

The most directly measurable result of the project is the reduction in preparation time already introduced in Chapter 1. Under the previous manual process, preparing the weekly reporting package required approximately one full working day, most of which was spent on repetitive extraction, cleaning, filtering, and consolidation tasks rather than on interpreting results. With the automated pipelines, the same reporting package can be generated in a maximum of approximately 30 minutes, covering the full sequence of running the relevant scripts through the GUI and reviewing the resulting Excel outputs.

Table 5.1: Reporting preparation time before and after automation

Reporting activity	**Manual process**	**Automated process**
Full weekly reporting package	∼1 working day	∼30 minutes

Source: Author

This reduction is consistent with the nature of the operations that were automated: data extraction, cleaning, filtering, cross-file matching, and calculation, which previously had to be repeated by hand for every reporting period, are now executed identically and near-instantaneously by the pipelines every time new source files are provided. The time freed by this reduction was, per the internship's stated objective, intended to be redirected toward interpretation of results, partner follow-up, and business decision-making rather than repetitive data preparation.

5.4 Sample Outputs

Figure 5.1 shows the dashboard sheet of a generated Weekly Partners Performance Report: KPI cards summarizing year-to-date orders, revenue, and target achievement, alongside per-partner achievement and revenue charts and the *Weekly Changes* panel introduced in Chapter 3. This is the output business users interact with directly, and illustrates concretely how the design decisions

discussed in the previous two chapters – conditional formatting, KPI cards, and in-place period-over-period deltas – come together in practice.

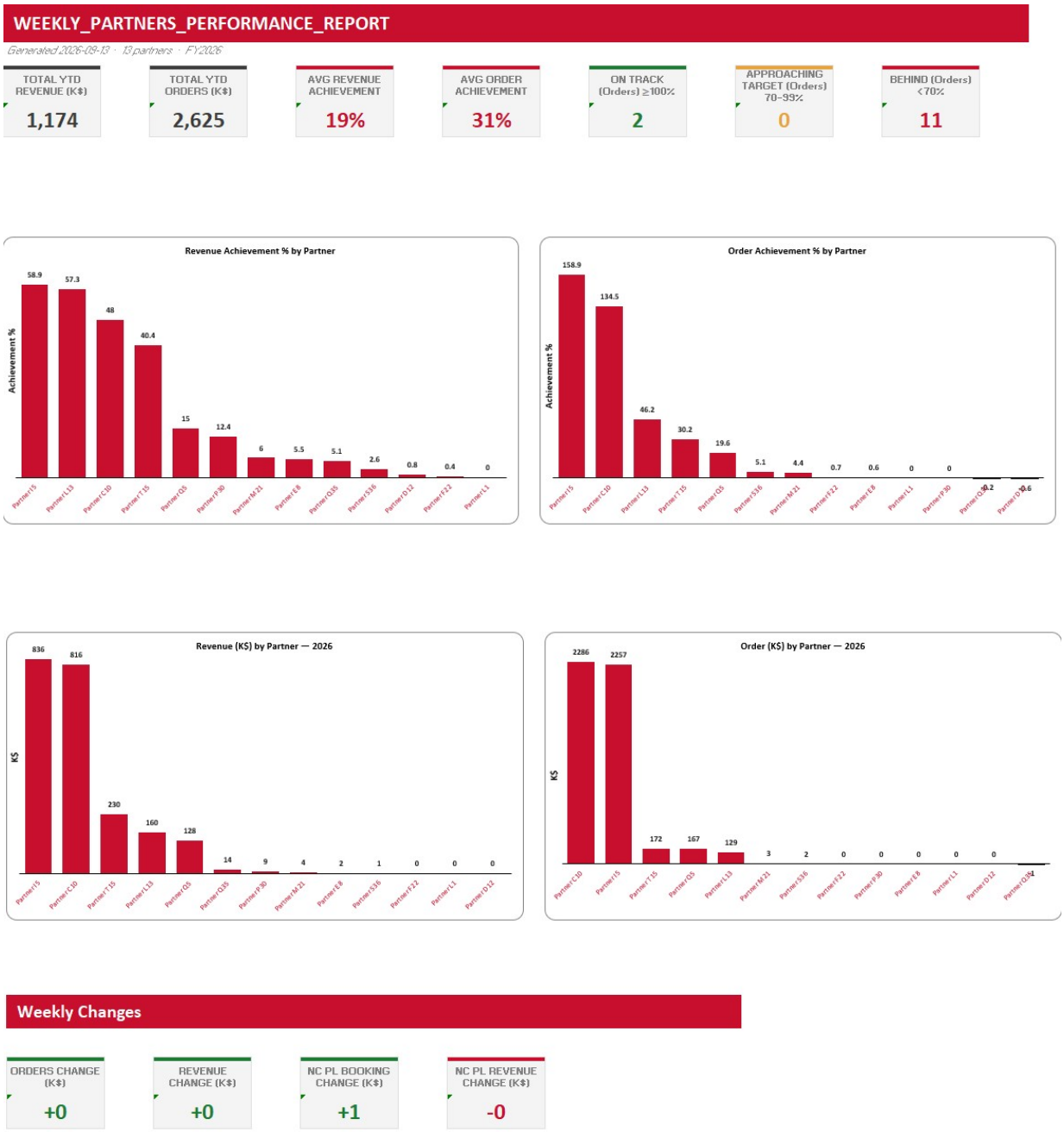


Figure 5.1: Weekly Partners Performance Report – dashboard sheet
Source: Author

Figure 5.2 shows the equivalent dashboard for the Certifications pipeline: KPI cards for certified partners, expiring certificates, new certificates, records needing review, and partner level-ups, alongside four charts covering partner level distribution, top partners by certificate count, expiration timing, and domain coverage. The color scheme is applied consistently with the other three dashboards, so all four reporting domains present a single, unified visual identity rather than four independently-styled outputs.

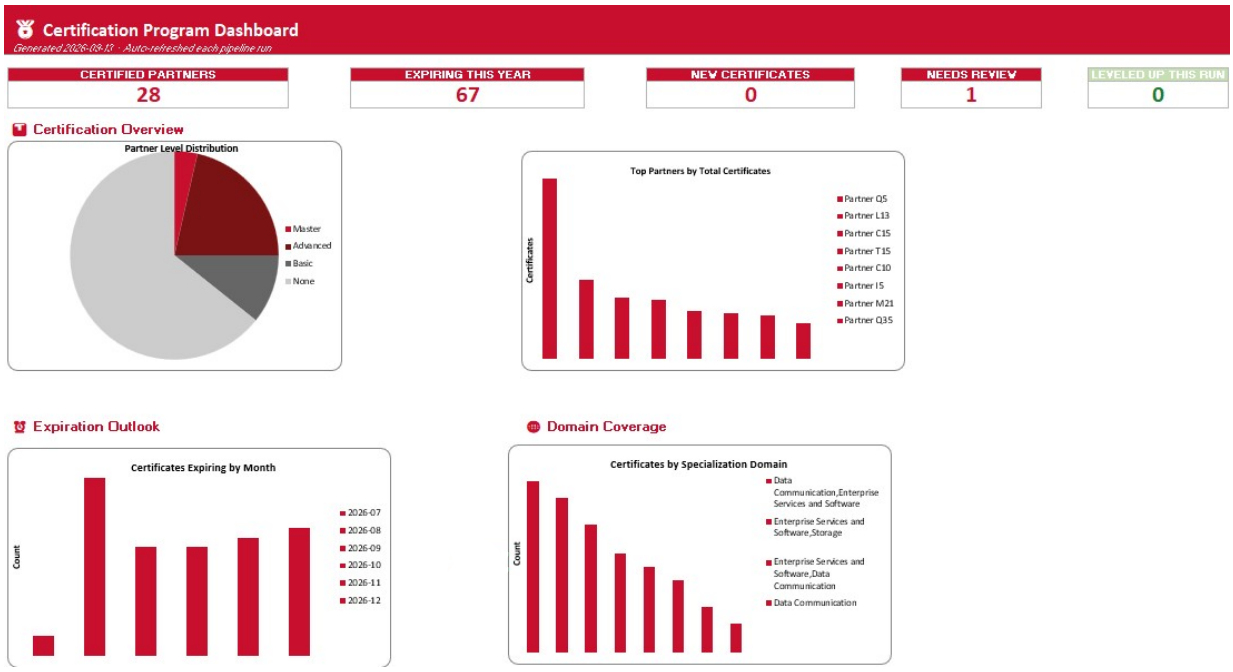


Figure 5.2: Certifications Report – dashboard sheet
Source: Author

5.5 Reliability and Data Quality

Beyond the aggregate time saving, the implementation includes several mechanisms specifically intended to make the automated output at least as reliable as the manual process it replaces, rather than only faster.

5.5.1 Duplicate and Data-Quality Handling

The Weekly pipeline's `sum_duplicates` function explicitly consolidates any partner appearing on more than one row of a raw export instead of silently dropping the duplicate row, and logs which partner codes were affected on every run. This directly addresses a data-quality issue identified during the analysis of the raw exports, and removes a class of error (undercounted totals) that a purely manual process would be equally exposed to, but without a systematic safeguard against it.

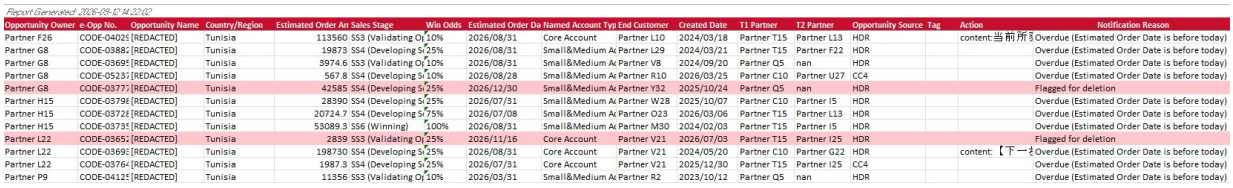
5.5.2 Review Flagging Instead of Silent Assumptions

Where a pipeline cannot confidently classify a record, it is designed to flag the record for manual review rather than guess. The Certifications pipeline's university-filtering step is the clearest example: any partner name not found in the editable `university_lookup.csv` table is routed to a separate *needs review* sheet instead of being silently kept or discarded, so that an unrecognized

entity cannot be misclassified without a human decision. The same principle applies to the Channel Revenue pipeline's Partner Type classification, where a raw contract row that does not match any of the defined Carrier/DEMO/STOCK/NC/Non-NC keyword rules falls through to *Direct* rather than being forced into one of the more specific categories.

5.5.3 Flagging Stale Records for Cleanup

The same principle extends to record-level data hygiene in the eSales pipeline. Figure 5.3 shows the Account Manager notification view, where each opportunity is checked against expected timeliness rules and, where relevant, annotated with a specific reason for example, an estimated order date that has already passed, or a record explicitly flagged for deletion rather than being left to age silently in the report. Surfacing a concrete reason for each flag, rather than a generic warning, allows the responsible Account Manager to act on the specific issue rather than having to re-investigate the record from scratch.



Report Generated: 2026-09-17 14:22:02

Opportunity Owner	e-Opp No.	Opportunity Name	Country/Region	Estimated Order Am	Sales Stage	Win Odds	Estimated Order Da	Named Account Typ	End Customer	Created Date	T1 Partner	T2 Partner	Opportunity Source	Tag	Action	Notification Reason
Partner F26	CODE-04025	[REDACTED]	Tunisia	113560	S3 (Validating Op	10%	2026/08/31	Core Account	Partner L10	2024/03/18	Partner T15	Partner L13	HDR		content:当前所处	Overdue (Estimated Order Date is before today)
Partner G8	CODE-03882	[REDACTED]	Tunisia	19873	S4 (Developing S	25%	2026/08/31	Small&Medium Ac	Partner L29	2024/03/21	Partner T15	Partner F22	HDR			Overdue (Estimated Order Date is before today)
Partner G8	CODE-03695	[REDACTED]	Tunisia	3974.6	S3 (Validating Op	10%	2026/08/31	Small&Medium Ac	Partner V8	2024/09/20	Partner Q5	nan	HDR			Overdue (Estimated Order Date is before today)
Partner G8	CODE-05237	[REDACTED]	Tunisia	567.8	S4 (Developing S	10%	2026/08/28	Small&Medium Ac	Partner R10	2026/03/25	Partner C10	Partner U27	CC4			Overdue (Estimated Order Date is before today)
Partner G8	CODE-03777	[REDACTED]	Tunisia	42585	S4 (Developing S	25%	2026/12/30	Small&Medium Ac	Partner Y32	2025/10/24	Partner Q5	nan	HDR			Flagged for deletion
Partner H15	CODE-03795	[REDACTED]	Tunisia	28390	S4 (Developing S	25%	2026/07/31	Small&Medium Ac	Partner W28	2025/10/07	Partner C10	Partner I5	HDR			Overdue (Estimated Order Date is before today)
Partner H15	CODE-03728	[REDACTED]	Tunisia	20724.7	S4 (Developing S	75%	2026/07/08	Small&Medium Ac	Partner O23	2026/03/06	Partner T15	Partner L13	HDR			Overdue (Estimated Order Date is before today)
Partner H15	CODE-03735	[REDACTED]	Tunisia	53089.3	S6 (Winning)	100%	2026/08/31	Small&Medium Ac	Partner M30	2024/02/03	Partner T15	Partner I5	HDR			Overdue (Estimated Order Date is before today)
Partner L22	CODE-03652	[REDACTED]	Tunisia	2839	S3 (Validating Op	25%	2026/11/16	Core Account	Partner V21	2026/07/03	Partner T15	Partner I25	HDR			Flagged for deletion
Partner L22	CODE-03695	[REDACTED]	Tunisia	198730	S4 (Developing S	25%	2026/08/31	Core Account	Partner V21	2024/05/20	Partner C10	Partner G22	HDR		content:【下一步	Overdue (Estimated Order Date is before today)
Partner L22	CODE-03764	[REDACTED]	Tunisia	1987.3	S4 (Developing S	25%	2026/07/31	Core Account	Partner V21	2025/12/30	Partner T15	Partner I25	CC4			Overdue (Estimated Order Date is before today)
Partner P9	CODE-04125	[REDACTED]	Tunisia	11356	S3 (Validating Op	10%	2026/03/31	Small&Medium Ac	Partner R2	2023/10/12	Partner Q5	nan	HDR			Overdue (Estimated Order Date is before today)

Figure 5.3: eSales pipeline – Account Manager notification view, flagging overdue and stale opportunities

Source: Author

5.5.4 Manual-Value Preservation

As described in Chapter 4, the Weekly and Channel Revenue pipelines carry forward manually-entered values (Status, channel, Partner AM, Huawei OO, Tier 1 PO fields) from the previous run's output whenever the current run's automated logic does not produce a value for that field. This directly addresses a risk specific to automation: that a fully regenerated report could silently erase business knowledge that only exists as a manual annotation, which would have made the automated version less reliable than the manual one it replaced rather than more.

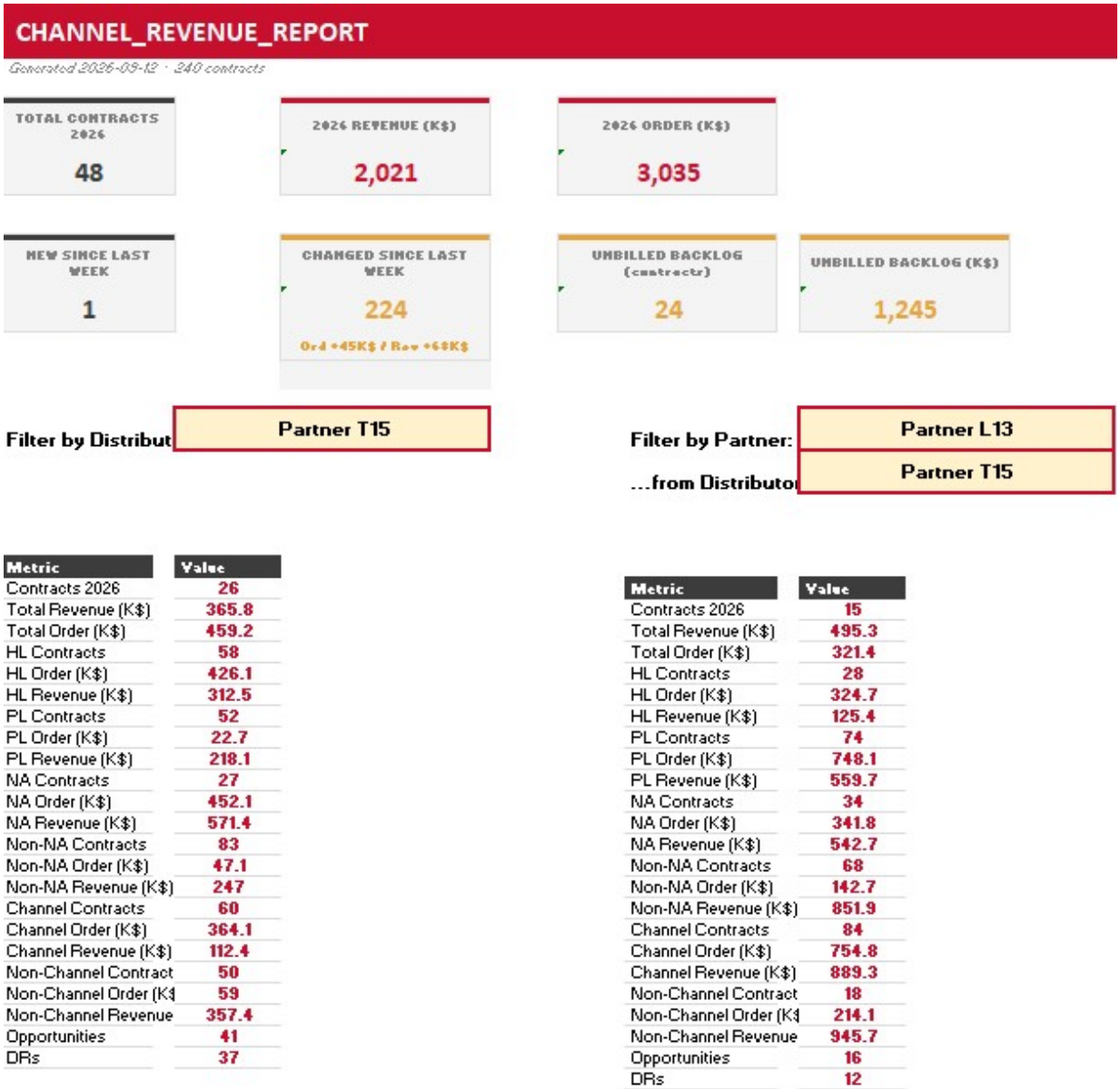
5.6 Business Use and Adoption

As presented in Chapter 1, the reporting system was put into use by the Channel Team during the internship period. By the end of the internship, the automated system had fully replaced the previous manual process for all four reporting domains, with business users relying on it directly for partner performance monitoring, business reviews, and partner meetings, without

requiring the business team to change how they read or interpreted the report, since the existing Excel-based reporting format was deliberately preserved rather than replaced.

This transition was shaped by iterative feedback gathered from the Channel Team throughout the month-long validation period described in Section 5.2. This feedback directly influenced several of the visual and layout decisions described in the dashboard design of Chapter 3 and the report styling of Chapter 4, reflecting the project's broader principle of developing the solution in continuous dialogue with its actual users rather than in isolation.

Figure 5.4 shows the resulting Channel Revenue Report dashboard, including the interactive distributor and sub-partner filter panels discussed in Chapter 3 – a direct product of this feedback cycle.



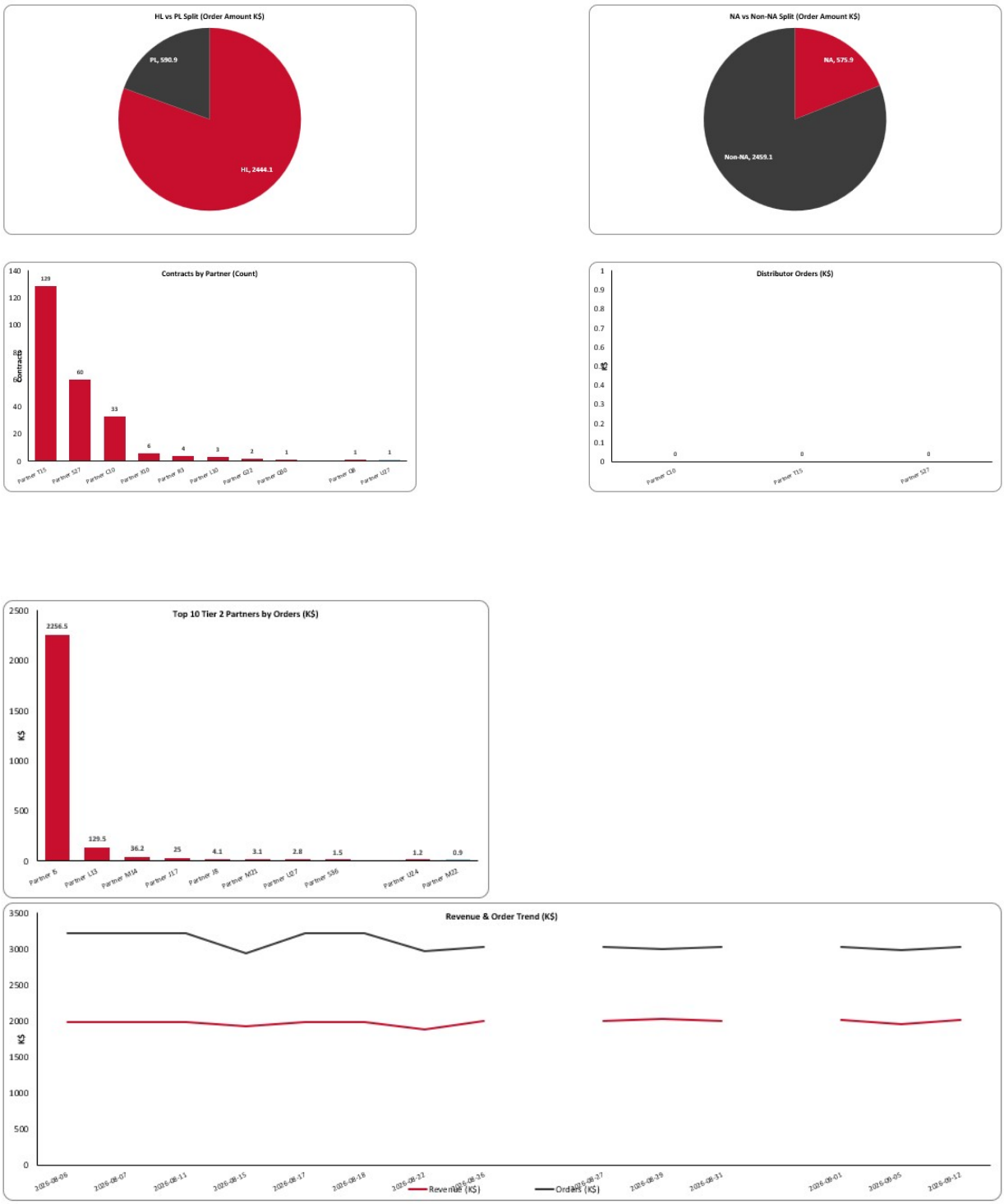


Figure 5.4: Channel Revenue Report – dashboard sheet with interactive filter panels
Source: Author

5.7 Limitations

The current implementation has several known limitations

5.7.1 Dependence on Source File Structure

Each pipeline expects its input files to follow a specific column structure and naming convention (for example, the bilingual Chinese/English headers of the raw DR export, or the fixed `PartnerCode`/`PartnerName` columns expected in the target and historical files). A structural change to any of these exports at the source system level would require a corresponding update to the relevant pipeline before it could process the new format correctly.

5.7.2 Fixed Partner and Classification Lists

Several business rules are currently expressed as fixed, manually maintained lists rather than being derived automatically for example, the seven partners included in the Weekly pipeline's NC Achievement table, the ten partners in the eSales pipeline's Ongoing Pipe table, and the keyword lists used for Partner Type and channel classification in the Channel Revenue pipeline. These lists are easy to review and edit as module-level constants, but they do require a manual update whenever the partner ecosystem changes (a new distributor onboarded, a partner renamed), rather than adapting automatically.

5.7.3 No Automated Test Suite

Validation of the pipelines currently relies on manually comparing automated output against previously produced manual reports, as described in the Evaluation Methodology section above, rather than on an automated regression test suite that would catch a logic error introduced by a future code change before it reaches a production report.

5.8 Future Work

Several extensions follow naturally from the limitations above and from the architectural choices discussed in Chapter 3 and Chapter 4.

Reducing reliance on fixed lists. Partner classification and scope lists (NC Achievement partners, Ongoing Pipe partners, Partner Type keywords) could be migrated from in-code constants to an external, structured reference file similar to the certifications pipeline's

`university_lookup.csv` and `partner_level_rules.csv`, making them editable by the business team without a code change.

Automated regression testing. A small suite of automated tests, built around a fixed set of anonymized sample inputs with known expected outputs, would allow future changes to the business logic to be validated automatically rather than relying solely on manual comparison against prior reports.

Growth toward a data-warehouse architecture. As discussed in Chapter 2, the current Excel-based, script-driven ETL pipeline was deliberately chosen over a full data-warehouse architecture because the Channel Team's present reporting needs (a small, known partner base, weekly rather than continuous reporting, a single primary report consumer) do not yet justify that additional complexity. Should the team's needs grow to include more partners, multiple concurrent report consumers, or a need for ad-hoc historical querying, the data-warehousing and OLAP literature reviewed in Chapter 2 (Chaudhuri and Dayal) provides the natural next architectural step.

5.9 Conclusion

This chapter evaluated the automated reporting system against the objectives defined at the start of the project. The solution achieved a substantial reduction in reporting preparation time, from approximately one working day to a maximum of approximately 30 minutes, while preserving the Excel-based reporting format already used by the Channel Team. Reliability was addressed directly in the implementation through duplicate handling, review-flagging of unclassifiable records, and preservation of manually-entered values across runs, rather than assumed as an automatic consequence of automation.

At the same time, the system retains clear limitations — dependence on stable source file structures, fixed classification lists requiring manual maintenance, incomplete historical depth for certain splits, and the absence of an automated test suite — which define concrete directions for future improvement rather than closed issues.

Overall, the project demonstrates that a lightweight, business-aligned automation approach, built around the team's existing tools and reporting formats rather than a larger BI platform, can deliver a substantial and immediately usable improvement to an organization's recurring reporting process.

General Conclusion

The internship provided an opportunity to combine business analytics, data processing, and process automation within a real professional environment. The project focused on improving the weekly reporting process of the Huawei Enterprise Business Group through the development of an automated reporting system.

The proposed solution integrates data extracted from existing Huawei systems, applies programmatic transformation and validation procedures, and consolidates the resulting information into comprehensive reports. The project therefore contributes to reducing repetitive manual work, improving reporting consistency, and facilitating the preparation of performance monitoring and business review activities.

Beyond the technical contribution, the internship also provided practical experience in partner enablement, certification coordination, training activities, event organization, stakeholder communication, and business performance analysis.

Future improvements may include further automation of data extraction, the integration of additional data sources, the development of interactive dashboards, and the implementation of more advanced analytical capabilities according to the evolving needs of the business team.

Bibliography

[1] Aleksandre Asatiani and Esko Penttinen. Turning robotic process automation into commercial success – case OpusCapita. *Journal of Information Technology Teaching Cases*, 6(2): 67–74, 2016. doi: 10.1057/jittc.2016.5.

[2] Hsinchun Chen, Roger H. L. Chiang, and Veda C. Storey. Business intelligence and analytics: From big data to big impact. *MIS Quarterly*, 36(4):1165–1188, 2012. doi: 10.2307/41703503.

[3] Stephen Few. *Information Dashboard Design: Displaying Data for At-a-Glance Monitoring*. Analytics Press, 2 edition, 2013. ISBN 9781938377006.

[4] Huawei. Huawei helps tunisia promote industry's digitalization development, 2018. URL `https://www.huawei.com/en/news/2018/7/Huawei-Tunisia-Digitalization-Development/`. Huawei's contribution to digital transformation in Tunisia.

[5] Huawei. Huawei 2025 annual report, 2026. URL `https://www.huawei.com/en/annual-report/2025`. Huawei annual report for fiscal year 2025.

[6] Huawei. Company facts, 2026. URL `https://www.huawei.com/en/media-center/company-facts`. Corporate fact sheet, updated April 2026.

[7] Huawei. Huawei global enterprise business partner program, 2026. URL `https://partner.huawei.com/?local=en`. Official Huawei Enterprise partner ecosystem information.

[8] Ralph Kimball and Joe Caserta. *The Data Warehouse ETL Toolkit: Practical Techniques for Extracting, Cleaning, Conforming, and Delivering Data*. Wiley, 2004. ISBN 978-0-7645-6757-5.

[9] Wes McKinney. Data structures for statistical computing in python. In *Proceedings of the 9th Python in Science Conference*, pages 56–61, 2010. doi: 10.25080/Majora-92bf1922-00a.

[10] Solomon Negash. Business intelligence. *Communications of the Association for Information Systems*, 13(1):177–195, 2004. doi: 10.17705/1CAIS.01315.

[11] Raymond R. Panko. What we know about spreadsheet errors. *Journal of End User Computing*, 10(2):15–21, 1998.

[12] Ken Peffers, Tuomas Tuunanen, Marcus A. Rothenberger, and Samir Chatterjee. A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3):45–77, 2007. doi: 10.2753/MIS0742-1222240302.

[13] Gloria E. Phillips-Wren, Mary Daly, and Frada Burstein. Reconciling business intelligence, analytics and decision support systems: More data, deeper insight. *Decision Support Systems*, 146:113560, 2021. doi: 10.1016/j.dss.2021.113560.

[14] Van H. Trieu. Getting value from business intelligence systems: A review and research agenda. *Decision Support Systems*, 93:111–124, 2017. doi: 10.1016/j.dss.2016.09.019.

[15] Panos Vassiliadis, Alkis Simitsis, and Spiros Skiadopoulos. On the logical modeling of etl processes. In *Proceedings of the 14th International Conference on Advanced Information Systems Engineering*, volume 2348 of *Lecture Notes in Computer Science*, pages 782–786. Springer, 2002. doi: 10.1007/3-540-47961-9_67.

[16] Richard Y. Wang and Diane M. Strong. Beyond accuracy: What data quality means to data consumers. *Journal of Management Information Systems*, 12(4):5–33, 1996. doi: 10.1080/07421222.1996.11518099.